Java OOP — Part 3.6: The java.lang Package in Depth and Design Capstone

Part 3.6 goes deep on the classes you use in every program: `String` and its builders, the wrapper classes, and the other important members of `java.lang`. It then closes the OOP track with a design capstone: principles and worked exercises that tie Parts 3.1 to 3.5 together.

Section 2.9 taught how to **use** strings, and 3.1.2 and 3.2 covered the keywords involved. This section covers how `String` **works inside** and the interview traps that come from it.

3.6.1 The String Class in Depth

`java.lang.String` is a `final`, immutable class that implements `CharSequence`, `Comparable<String>`, `Serializable` and `Constable`. Everything below follows from three facts: it is **immutable**, it is **shared** (the pool), and it is **the most common hash-map key** in Java.

How a String is stored

```
// Simplified view of java.lang.String (Java 9 and later)
public final class String {
    private final byte[] value;        // the characters
    private final byte coder;          // 0 = LATIN1 (1 byte per char), 1 =
UTF16 (2 bytes per char)
    private int hash;                  // cached hashCode
    private boolean hashIsZero;        // true if the hash really is 0 (so it
is not recomputed)
}
```

- **Compact strings (Java 9):** if every character fits in Latin-1 (code points 0 to 255, which covers English and most Western European text), the text is stored **one byte per character**; otherwise two bytes (UTF-16). Before Java 9 every string used a `char[]`, two bytes per character. This often cuts heap use noticeably for typical text, and it is invisible to your code (`length()` and `charAt()` still count UTF-16 code units, 1.3). It can be switched off with `-XX:-CompactStrings`.
- A `String` is **two objects**: the `String` object (header, the `value` reference, `coder`, `hash`) and the `byte[]` it points to. Each carries an object header, so a short string costs far more than its characters.

- **`substring`** copies the characters since Java 7u6. In earlier versions it shared the parent's array, so keeping a tiny substring of a huge string kept the huge array alive, a famous memory leak that was fixed by copying.

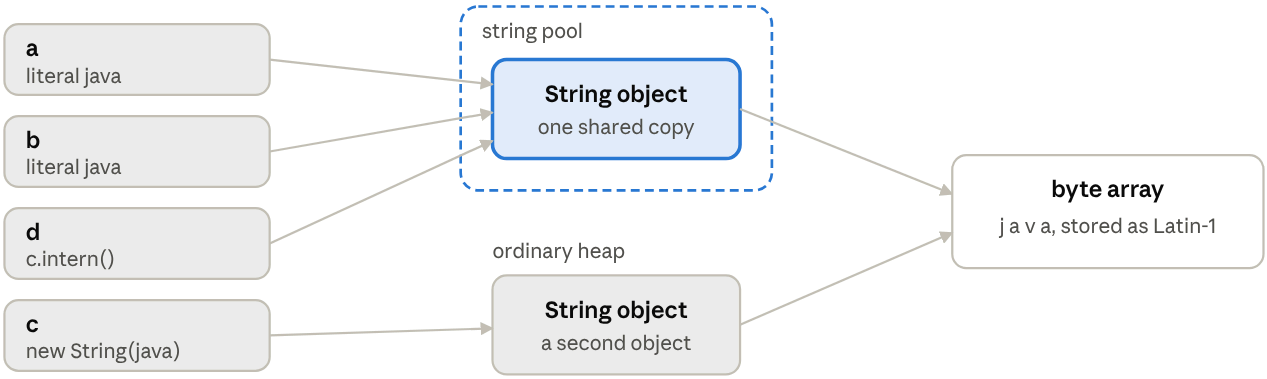
The string pool

The JVM keeps a table of unique strings, the **string pool** (the *string table*), so identical literals are the same object.

```
String a = "java";                 // literal: looked up in the pool
String b = "java";                 // same pooled object
String c = new String("java");     // a NEW String object (the literal is
also in the pool)
String d = c.intern();             // the pooled instance

System.out.println(a == b);        // true
System.out.println(a == c);        // false
System.out.println(a == d);        // true
System.out.println(a.equals(c));   // true
```

Three variables share the pooled String; new String makes another



a == b and a == d are true. a == c is false, because new String creates a second String object.
The two String objects still share one byte array, so the characters are stored only once.

String pool, new String and intern

Facts about the pool:

- Literals (and compile-time constant expressions like `"ja" + "va"`, and `final` variables with constant values) go into the pool when the class's constant is first resolved by the JVM.

- Since **Java 7** the pooled objects live on the **ordinary heap**, so unused interned strings can be garbage collected (before, they sat in PermGen). The pool's hash table is native memory of a fixed number of buckets, tunable with `-XX:StringTableSize`.
- `intern()` returns the canonical pooled instance, adding the string if it is not there yet. It costs a table lookup; use it only when you have very many long-lived duplicate strings.
- `new String(String original)` creates a second `String` object but **shares the same byte array** as the original, so the characters are stored once; the extra cost is one small object.
- **String deduplication:** G1, ZGC and Shenandoah can deduplicate the **byte arrays** of equal strings in the background (`-XX:+UseStringDeduplication`), without changing identity.

```
String x = "ja" + "va";                 // constant expression: folded to
"java" at compile time
System.out.println(a == x);             // true
String p = "ja";
String y = p + "va";                    // computed at run time: a new object
System.out.println(a == y);             // false
final String q = "ja";
System.out.println(a == q + "va");      // true: q is a constant variable
```

Creating strings

Way	Object created
`"text"`	The pooled instance (created once)
`new String("text")`	A new `String`; almost never needed (the useful case is `new String(bytes, UTF_8)`)
`String.valueOf(x)`, `Integer.toString(n)`	A new `String`
`sb.toString()`	A new `String` (copies the builder's characters)
`"a" + variable`	A new `String` at run time
Text block `"""..."""`	A pooled constant

hashCode, equals and compareTo

The hash is computed once from the characters and **cached**:

```
hashCode = s[0]*31^(n-1) + s[1]*31^(n-2) + ... + s[n-1]
```

```
System.out.println("Aa".hashCode());      // 2112
System.out.println("BB".hashCode());      // 2112   different strings, same
hash: a collision
```

Because strings are immutable the cached hash can never go stale, which is why `String` is a perfect `HashMap` key. Predictable hashes let attackers craft many colliding keys (a *hash flooding* attack); `HashMap` limits the damage by turning long bucket chains into balanced trees, and `String` is `Comparable` so those trees work well (3.5.2).

`equals` compares the lengths and then the bytes (and the coder) and is intrinsified by the JIT into very fast vector code. `compareTo` compares lexicographically by UTF-16 code unit values and returns the difference of the first differing characters, or the length difference.

```
"apple".compareTo("apricot");        // negative: 'p' < 'r' at index 2
"b".compareTo("abc");                // positive
"abc".compareTo("abcd");             // -1: a prefix is smaller
"Hello".equalsIgnoreCase("hELLO");   // true
"a".contentEquals(new StringBuilder("a"));   // true: compares with any
CharSequence
```

Locale and Unicode traps

- `toUpperCase()` and `toLowerCase()` use the **default locale**. In a Turkish locale `"TITLE".toLowerCase()` gives a dotless `i` and breaks comparisons. For identifiers, protocol keywords and keys use **`toLowerCase(Locale.ROOT)`**.
- `equalsIgnoreCase` is a simple per-character comparison, fine for ASCII and most uses, but it does not treat `"é"` written as two code points (e + combining accent) as equal to the precomposed one. Normalize with `java.text.Normalizer` and use `java.text.Collator` for human-facing sorting.
- `length()` counts UTF-16 code units. Use `codePointCount` and `codePoints()` for emoji and other supplementary characters (1.3).
- `String.format` and `printf` also use the default locale (decimal comma in some locales); pass `Locale.ROOT` or `Locale.US` for machine-readable output.

How `+` really works

Java 8 compiled `a + b + c` into `new StringBuilder().append(a).append(b).append(c).toString()`. Since **Java 9** `javac` emits one `invokedynamic` call to `StringConcatFactory`, which sizes the result exactly and builds it in one pass. Either way, concatenating **in a loop** is still quadratic, because each iteration creates a new string from the old one; use a `StringBuilder` (3.6.2).

```
String s = null;
System.out.println("v=" + s);                  // v=null     (null becomes
the text "null")
System.out.println(1 + 2 + "x" + 1 + 2);       // 3x12
System.out.println(String.valueOf((Object) null));   // null
// System.out.println(String.valueOf(null));    // NullPointerException:
resolves to valueOf(char[])
```

Regular-expression methods and their cost

`split`, `matches`, `replaceAll`, `replaceFirst` take **regular expressions**. `split` has a fast path for a single non-regex character (`"a,b".split(",")`), but `replaceAll` and `matches` **compile a `Pattern` on every call**. In hot code, precompile:

```
private static final Pattern DIGITS = Pattern.compile("\\d+");
String clean = DIGITS.matcher(text).replaceAll("#");
// "a.b".split(".") returns an EMPTY array: '.' is a regex that matches every
character; use split("\\.") or Pattern.quote
```

API additions by version

Version	Additions
5 to 8	`format`, `contains(CharSequence)`, `isEmpty`, `String.join`, `chars()`, switch on strings (7)
9	**Compact strings** (internal)
11	`isBlank`, `lines`, `strip`, `stripLeading`, `stripTrailing`, `repeat`
12	`indent`, `transform`

Version	Additions
15	**Text blocks** (`"""`), `formatted` , `stripIndent` , `translateEscapes`
21	`indexOf(ch, from, to)` , `splitWithDelimiters` ; `StringBuilder.repeat`

```
"  hi  ".strip();                   // "hi"    Unicode-aware (trim() only
handles chars <= U+0020)
"".isBlank();                       // true
"ab".repeat(3);                     // "ababab"
"x\ny".lines().toList();            // [x, y]
"%s is %d".formatted("Asha", 30);   // "Asha is 30"
"hello".transform(String::toUpperCase);   // "HELLO"
```

Security and memory notes

- **Passwords:** a `String` stays in memory until garbage collected and may be pooled or appear in heap dumps, and cannot be wiped. APIs such as `Console.readPassword` return a `char[]` that you can overwrite with `Arrays.fill` after use.
- **Large text:** a big `String` is one big object; process streams (`Files.lines` , `BufferedReader`) instead of loading everything.
- **Avoid `intern()` on unbounded or user-supplied input**, since it grows a shared table.
- Do not use `String` for things that have a type (a date, a number, an id): wrap them in value types (records).

Interview traps

- *Why is `String` immutable? Why final?* Security, thread safety, cached hash, pooling; `final` stops a subclass from adding mutability.
- *How many objects does `String s = new String("abc")` create?* Up to two (the pooled literal if new, plus the heap object), plus the shared array.
- *What does `intern()` do?* Returns the canonical pooled instance, adding it if absent.
- *Where is the string pool?* On the heap since Java 7 (PermGen before).
- *Is `"a" + "b" == "ab"` ?* `true` : constant folding.
- *`equals` vs `==` on strings?* `equals` compares contents; `==` compares identity and only works by accident for pooled values.

- *What changed in Java 9?* Compact strings (`byte[]` plus a coder) and `invokedynamic` concatenation.
- *Why `char[]` for passwords?* It can be cleared; a string stays in memory until collected.

3.6.2 StringBuilder vs StringBuffer

Because `String` is immutable, building text piece by piece with `+` creates a new string at every step (3.6.1). A **builder** is a **mutable** sequence of characters that you modify in place and convert to a `String` once at the end.

	`String`	`StringBuilder`	`StringBuffer`
Mutable	No	**Yes**	**Yes**
Thread-safe	Yes (immutable)	**No**	**Yes** (each method is `synchronized`)
Speed for repeated edits	Slow	**Fastest**	Slower (locking)
Since	1.0	Java 5	1.0
`equals` compares contents?	Yes	**No** (identity)	**No** (identity)
Use	Fixed text, keys, constants	**Default for building text**	Legacy code that shares one buffer between threads

How a builder works inside

`StringBuilder` and `StringBuffer` both extend the package-private `AbstractStringBuilder`, which holds the data:

```
// Simplified
abstract class AbstractStringBuilder {
    byte[] value;      // the buffer (Latin-1 or UTF-16 bytes, as in String)
    byte coder;        // switches to UTF-16 if a non-Latin-1 character is
appended
    int count;         // how many characters are in use (length())
}
```

- The buffer's size is the **capacity**; the used part is the **length**. Appending fills spare capacity with no new allocation.
- When the buffer is full it grows to about **twice the old capacity plus 2** (`(old << 1) + 2`) and the contents are copied. Because growth is geometric, appending is **amortized O(1)** per character.
- Default capacity is **16**; `new StringBuilder("abc")` starts with `3 + 16 = 19`; `new StringBuilder(n)` starts with `n` .

```
StringBuilder sb = new StringBuilder();          // capacity 16
sb.append("x".repeat(17));                       // 17 > 16: grows to (16 <<
1) + 2 = 34
System.out.println(sb.capacity());               // 34
System.out.println(sb.length());                 // 17
sb.ensureCapacity(1000);                         // reserve space in advance
sb.trimToSize();                                 // release unused capacity
```

Pre-size when you know roughly how much you will write (`new StringBuilder(8192)`): it avoids repeated copying.

The main methods

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" ").append(21).append('!').append(3.5).append(true);   // Java
21!3.5true   each call returns 'this'
sb.insert(0, ">> ");              // ">> Java 21!3.5true"     shifts
everything right: O(n)
sb.setCharAt(3, 'j');             // change one character in place
sb.deleteCharAt(sb.length() - 1); // remove one character
sb.delete(0, 3);                  // remove a range: [start, end)
sb.replace(0, 4, "JAVA");         // replace a range with a string
sb.reverse();                     // reverses the sequence (treats surrogate
pairs as one character)
sb.setLength(0);                  // empty the builder and reuse it (no
reallocation)
int i = sb.indexOf("va");         // search, like String
String s = sb.toString();         // copies the characters into a NEW String
sb.repeat("ab", 3);               // Java 21: appends "abababab"
sb.isEmpty();                     // Java 15
```

Method	Notes
`append(x)`	Overloads for every primitive, `char[]`, `String`, `CharSequence` (and ranges), `Object`; appends `"null"` for a `null` reference
`insert(offset, x)`	Same overloads; O(n) because the tail moves
`delete(start, end)`, `deleteCharAt(i)`	End is exclusive; O(n)
`replace(start, end, str)`	Replaces a range
`reverse()`	In place; surrogate-pair aware
`setLength(n)`	Truncates, or pads with `'\0'` if `n` is larger
`charAt`, `setCharAt`, `length`, `capacity`, `substring`, `indexOf`	As expected
`toString()`	Allocates a new `String`; the builder is unchanged

Method	Notes
`compareTo`	Since Java 11, lexicographic comparison of the contents

Traps

```
StringBuilder a = new StringBuilder("x"), b = new StringBuilder("x");
System.out.println(a.equals(b));                    // false: builders do
NOT override equals
System.out.println(a.toString().equals(b.toString())); // true
System.out.println(a.compareTo(b));                 // 0
System.out.println(a.contentEquals(b) );            // compile error: that
method is on String; use toString().contentEquals(b)

StringBuilder sb = new StringBuilder();
sb.append('a' + 1);                                 // appends "98": 'a' + 1
is an int
sb.append((char) ('a' + 1));                        // appends "b"
sb.append((String) null);                           // appends "null"

StringBuilder bad = new StringBuilder(65);          // capacity 65, NOT the
character 'A'
```

- A `StringBuilder` is **not** a `String`: you cannot pass it where a `String` is required; call `toString()`. It **is** a `CharSequence`, so APIs that take `CharSequence` (such as `String.contains`, `Pattern.matcher`) accept it directly.
- `new StringBuilder(char)` does not exist; passing a `char` selects the `int` capacity constructor.
- `insert(0, x)` in a loop is O(n²); build in order or use a `Deque` and join.
- Builders do not implement `hashCode` by content, so they are poor map keys.

Why builders beat `+` in a loop

```
String s = "";
for (int i = 0; i < 100_000; i++) s += i;            // 100,000 intermediate
strings, each copying all previous text: O(n^2)

StringBuilder sb = new StringBuilder(600_000);
for (int i = 0; i < 100_000; i++) sb.append(i);      // one buffer, appended
in place: O(n)
String result = sb.toString();
```

A single expression such as `"a" + x + "b"` is already compiled efficiently (3.6.1). The builder matters when the concatenation is **repeated** across iterations or branches.

Thread safety: why StringBuffer exists, and why it rarely helps

`StringBuffer` marks each public method `synchronized`, so one call is atomic with respect to others on the same object. That does **not** make a **sequence** of calls atomic:

```
StringBuffer shared = new StringBuffer();
// Thread A:                          // Thread B:
shared.append("user=");               shared.append("admin");
shared.append("alice");               shared.append(";");
// The result can interleave: "user=adminalice;" The individual appends were
safe, the sequence was not.
```

- Most code builds a string in a **local variable** and never shares it, so locking is pure overhead. Use **`StringBuilder`**.
- If several threads must contribute, either **confine** each to its own builder and combine later, or guard the whole sequence with your own lock, or use a thread-safe structure (a `ConcurrentLinkedQueue<String>` joined at the end, or a logging framework's appender).
- HotSpot can **elide** the lock when a `StringBuffer` provably never leaves its method (escape analysis), so old code is not as slow as it looks, but there is no reason to choose it in new code.
- `StringBuffer` remains in some legacy APIs; since Java 9 most of them also accept `StringBuilder` (for example `Matcher.appendReplacement`).

Alternatives for joining text

```
String csv = String.join(",", List.of("a", "b", "c"));
// a,b,c
String out = items.stream().map(Item::name).collect(Collectors.joining(", ",
"[", "]"));   // [x, y, z]

StringJoiner sj = new StringJoiner(", ", "{", "}");     // delimiter, prefix,
suffix
sj.setEmptyValue("{}");                                 // what to return
when nothing was added
sj.add("a").add("b");
System.out.println(sj);                                 // {a, b}

String line = "%s scored %d".formatted("Asha", 98);      // formatted text
StringWriter sw = new StringWriter();                    // build output
through the Writer API
try (PrintWriter pw = new PrintWriter(sw)) { pw.printf("%5.2f%n", 3.14159); }
```

Use `String.join` or `Collectors.joining` for a delimited list, `StringJoiner` for incremental joining with prefix and suffix, `String.format` or `formatted` for templates, and a builder for everything that is assembled in steps.

Practical guidance

- **Build SQL with parameters, not concatenation.** Appending user input into a query string is an SQL-injection hole; use `PreparedStatement`.
- **Do not build log messages eagerly.** `log.debug("x=" + big)` allocates even when debug is off; use `log.debug("x={}", big)`.
- **Do not keep a huge builder alive.** Its buffer stays at the largest capacity reached; `trimToSize()` or let it go out of scope.
- **Reuse** a builder in a loop with `setLength(0)` instead of allocating a new one each iteration.
- Reversing: `new StringBuilder(s).reverse().toString()` is correct for emoji and other supplementary characters.

Interview traps

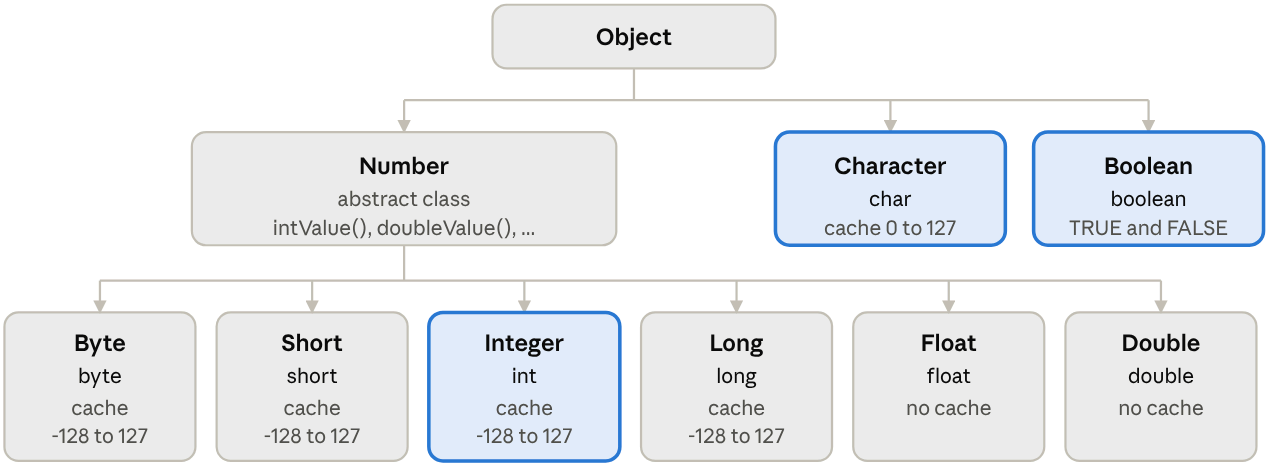
- *`StringBuilder` vs `StringBuffer`?* Same API; `StringBuffer` is synchronized and slower; `StringBuilder` is the default.

- *Is `StringBuffer` enough for thread-safe string building?* Each call is atomic, a sequence of calls is not.
- *What is the default capacity and how does it grow?* 16; roughly doubles (`(old << 1) + 2`).
- *Does `sb1.equals(sb2)` compare content?* No; compare `toString()` values or use `compareTo` .
- *How many objects does `sb.toString()` create?* One new `String` (plus its byte array); the builder keeps its own buffer.
- *Why does `sb.append('a' + 1)` print 98?* `'a' + 1` is an `int` , so `append(int)` is chosen.
- *When is `+` fine?* A single expression, or compile-time constants; avoid it when repeated in a loop.

3.6.3 Wrapper Classes

Each of the eight primitive types has a **wrapper class** in `java.lang` that holds a primitive value inside an object. Section 2.4 introduced them briefly; here are the internals, the traps and the utility methods.

Eight wrappers: six extend Number, two extend Object directly



Integer.valueOf(127) == Integer.valueOf(127) is true; with 128 it is false, so compare boxed values with equals.
BigInteger, BigDecimal and AtomicInteger also extend Number, but they are not wrappers of a primitive.
All wrapper classes are final and immutable.

The wrapper classes: hierarchy and caches

Primitive	Wrapper	Parent	Notes
`byte`	`Byte`	`Number`	Cached: -128 to 127
`short`	`Short`	`Number`	Cached: -128 to 127
`int`	`Integer`	`Number`	Cached: -128 to 127 (upper bound tunable)
`long`	`Long`	`Number`	Cached: -128 to 127
`float`	`Float`	`Number`	No cache
`double`	`Double`	`Number`	No cache
`char`	`Character`	`Object`	Cached: 0 to 127
`boolean`	`Boolean`	`Object`	Two shared instances, `TRUE` and `FALSE`

Why wrappers exist

- **Generics and collections hold objects**: `List<Integer>`, never `List<int>`.
- A primitive **cannot be `null`**; a wrapper can represent "no value" (an absent database column).
- They provide **utility methods and constants**: parsing, formatting, comparison, bit operations, `MAX_VALUE`.
- They allow primitives to be used as `Object` (a key, a lock, a method argument of type `Object`).
- All are **`final`**, **immutable**, `Comparable`, and `Serializable`.

Autoboxing and unboxing

Since Java 5 the compiler converts automatically between a primitive and its wrapper.

```
Integer a = 10;            // autoboxing:  Integer a = Integer.valueOf(10);
int b = a;                 // unboxing:    int b = a.intValue();

List<Integer> list = new ArrayList<>();
list.add(5);               // boxing
int first = list.get(0);   // unboxing

Integer count = 0;
count++;                   // count = Integer.valueOf(count.intValue() + 1):
a NEW Integer object (they are immutable)
```

Boxing uses `valueOf` (which can return a cached object) and unboxing uses the `xxxValue()` method. It happens in assignments, method arguments, arithmetic, collections, comparisons with `<`, and conditional expressions.

The caches

`Integer.valueOf(int)` returns a **shared** object for values from -128 to 127, and creates a new one otherwise. The same idea applies to `Byte`, `Short`, `Long`, `Character` (0 to 127) and `Boolean`. The upper limit of the `Integer` cache can be raised with `-XX:AutoBoxCacheMax=<n>`. `Float` and `Double` have **no cache**.

```
Integer x = 127, y = 127;
System.out.println(x == y);                // true  (same cached object)
Integer p = 128, q = 128;
System.out.println(p == q);                // false (two different objects)
System.out.println(p.equals(q));           // true

System.out.println(Double.valueOf(1.0) == Double.valueOf(1.0));    // false:
no cache
System.out.println(Boolean.valueOf(true) == Boolean.TRUE);         // true
// new Integer(5) is deprecated for removal: it always allocates; use
Integer.valueOf(5) or autoboxing
```

Rule: compare boxed values with `equals`, or unbox them (`a.intValue() == b.intValue()`), never with `==`.

Traps

1. `equals` is type-strict.

```
Long l = 5L;
System.out.println(l.equals(5));           // false: 5 is boxed to an
Integer, and Long.equals(Integer) is false
System.out.println(l.equals(5L));          // true
System.out.println(new HashSet<>(List.of(1L)).contains(1));   // false:
Integer 1 is not Long 1L
```

2. Unboxing `null` throws `NullPointerException`.

```
Integer count = map.get("missing");        // null
int n = count;                             // NullPointerException

Integer a = null;
boolean flag = true;
Integer r = flag ? a : 0;                  // NullPointerException!
// One branch is Integer, the other is int, so the conditional unboxes `a`
(binary numeric promotion).
Integer r2 = flag ? a : Integer.valueOf(0);   // fine: both branches are
Integer
```

3. Overload resolution: `remove(int)` vs `remove(Object)`.

```
List<Integer> nums = new ArrayList<>(List.of(10, 20, 30));
nums.remove(1);                            // removes the element at INDEX 1
-> [10, 30]
nums.remove(Integer.valueOf(10));          // removes the VALUE 10
-> [30]

Set<Short> set = new HashSet<>();
for (short i = 0; i < 5; i++) { set.add(i); set.remove(i - 1); }   // i - 1
is an int, boxed to Integer: never matches a Short
System.out.println(set.size());            // 5, not 1
```

4. Comparison operators unbox, `==` does not. `a < b` unboxes both, but `a == b` on two `Integer` references compares identity (unless one operand is a primitive, in which case it unboxes).

5. Silent widening. A wrapper does not widen like a primitive: `Long x = 5;` does not compile (an `int` cannot be boxed to `Long`); write `5L`. `Object o = 5; long v = (long) o;` throws `ClassCastException`.

6. `Double` and `-0.0` / `NaN`. `Double.equals` compares the bit patterns, so `Double.valueOf(0.0).equals(-0.0)` is `false` and `Double.NaN` equals itself; with primitives `0.0 == -0.0` is `true` and `NaN == NaN` is `false` (2.4, 3.5.3).

Performance and memory

```
Long sum = 0L;                                      // BAD in a hot loop
for (long i = 0; i < 1_000_000; i++) sum += i;      // each += unboxes, adds,
and allocates a new Long

long total = 0;                                     // GOOD
for (long i = 0; i < 1_000_000; i++) total += i;
```

- An `Integer` object takes about 16 bytes (header + value) against 4 bytes for an `int`, and a `List<Integer>` also stores a 4 or 8 byte reference per element: roughly **5 times the memory** and poorer cache locality than an `int[]`.
- Boxing creates garbage; the JIT removes some of it with escape analysis, but you should not rely on that in hot paths.
- Use **primitive streams** (`IntStream`, `mapToInt(...).sum()`), **primitive arrays**, and specialized collections (Eclipse Collections, fastutil) for large numeric data. Project Valhalla aims to reduce this overhead with value classes; check the current JEP status for what has shipped.

Converting: parse, valueOf, toString

```
int n  = Integer.parseInt("123");           // String to int       ("12a", ""
or null -> NumberFormatException)
Integer w = Integer.valueOf("123");        // String to Integer   (uses the
cache)
int h  = Integer.parseInt("ff", 16);       // 255: with a radix
int d  = Integer.decode("0x1F");           // 31: understands 0x, 0, #
String s = Integer.toString(255, 2);       // "11111111"
String t = String.valueOf(255);            // "255"
double x = Double.parseDouble("3.5e2");    // 350.0
boolean f = Boolean.parseBoolean("yes");   // false: only "true" (any case)
is true
int digit = Character.getNumericValue('7');// 7
```

Useful members

Class	Members worth knowing
`Integer`, `Long`	`MAX_VALUE`, `MIN_VALUE`, `SIZE`, `BYTES`, `compare`, `signum`, `sum`, `max`, `min`, `bitCount`, `reverse`, `highestOneBit`, `numberOfLeadingZeros`, `toBinaryString`, `toHexString`, `toOctalString`, `parseUnsignedInt`, `divideUnsigned`, `compareUnsigned`
`Double`, `Float`	`isNaN`, `isInfinite`, `isFinite`, `compare`, `MIN_VALUE` (smallest **positive** value, not the most negative), `MAX_VALUE`, `doubleToLongBits`
`Character`	`isDigit`, `isLetter`, `isLetterOrDigit`, `isAlphabetic`, `isUpperCase`, `isWhitespace`, `toUpperCase`, `getNumericValue`, `toString(int codePoint)`, `charCount`, `isEmoji` (21)
`Boolean`	`parseBoolean`, `logicalAnd`, `logicalOr`, `logicalXor`, `TRUE`, `FALSE`
`Number` (abstract)	`intValue`, `longValue`, `floatValue`, `doubleValue`, `byteValue`, `shortValue`; these **narrow** like casts

Hash codes: `Integer.hashCode()` is the value; `Long` folds the two halves (`(int)(v ^ (v >>> 32))`); `Double` uses its bits; `Boolean` returns 1231 or 1237; `Character` is its code unit.

BigInteger and BigDecimal

Both extend `Number` but are not primitive wrappers: they are **immutable arbitrary-precision** types in `java.math`.

```
BigInteger f = BigInteger.ONE;
for (int i = 2; i <= 30; i++) f = f.multiply(BigInteger.valueOf(i));   // 30! does not fit in a long
System.out.println(f);                    // 265252859812191058636308480000000
BigInteger big = new BigInteger("12345678901234567890123456789");
big.mod(BigInteger.valueOf(97));  big.pow(3);  big.isProbablePrime(20);
```

`BigDecimal` rules (see 2.4 and 3.5.3): build from a `String` or `valueOf`, use `compareTo` not `equals` for numeric equality, always give a scale and `RoundingMode` when dividing, and remember every operation returns a new object.

Interview traps

- *Output of `Integer a = 127, b = 127; a == b` and with 128?* `true`, then `false` (cache).
- *Why does `list.remove(1)` on a `List<Integer>` remove by index?* Overload resolution prefers the exact `int` match `remove(int)` over boxing.
- *What is autoboxing and what code does the compiler generate?* `valueOf` for boxing, `xxxValue()` for unboxing.
- *Which wrappers are not cached?* `Float` and `Double`.
- *Can unboxing throw?* Yes, `NullPointerException` on `null`.
- *Why prefer `int` to `Integer` in loops and fields?* Less memory, no allocation, faster, and no `null`.
- *Is `Long.equals(Integer)` ever true?* No; the types must match.
- *Are wrappers mutable?* No; `i++` on an `Integer` creates a new object and re-points the variable.

3.6.4 Other Important Classes of java.lang

Everything in `java.lang` is **imported automatically**. Besides `Object`, `String`, the builders and the wrappers, these are the classes and interfaces you meet most often.

Group	Members
Numbers and math	`Math`, `StrictMath`, `Number`, `Character`
Environment	`System`, `Runtime`, `Process`, `ProcessBuilder`, `ProcessHandle`, `StackWalker`
Errors	`Throwable`, `Exception`, `RuntimeException`, `Error` and their subclasses
Threads	`Thread`, `Runnable`, `ThreadLocal`, `ScopedValue`
Reflection and types	`Class`, `ClassLoader`, `Module`, `Package`, `Enum`, `Record`, `Void`

Group	Members
Core interfaces	`Comparable`, `CharSequence`, `Iterable`, `AutoCloseable`, `Cloneable`, `Appendable`, `Runnable`
Annotations	`@Override`, `@Deprecated`, `@SuppressWarnings`, `@SafeVarargs`, `@FunctionalInterface`

Math and StrictMath

`Math` is a `final` class of static methods (it has a private constructor). Constants: `Math.PI`, `Math.E`.

```
Math.abs(-5);  Math.max(3, 9);  Math.min(3, 9);
Math.pow(2, 10);            // 1024.0   (always double)
Math.sqrt(16);              // 4.0       Math.cbrt(27) = 3.0
Math.hypot(3, 4) = 5.0
Math.floor(-2.5);           // -3.0      Math.ceil(-2.5) = -2.0
Math.round(2.5);            // 3         (long for a double, int for a float;
rounds half UP, so round(-2.5) is -2)
Math.rint(2.5);             // 2.0       (rounds half to EVEN)
Math.log(Math.E);  Math.log10(1000);  Math.exp(1);  Math.sin(Math.PI / 2);
```

The `*Exact` family throws `ArithmeticException` instead of silently overflowing; use it for money, counts and sizes.

```
Math.addExact(Integer.MAX_VALUE, 1);     // ArithmeticException: integer
overflow
Math.multiplyExact(100_000, 100_000);    // ArithmeticException
Math.toIntExact(3_000_000_000L);         // ArithmeticException (a cast would
give a wrong number)
Math.absExact(Integer.MIN_VALUE);        // ArithmeticException (Java 15);
Math.abs would stay negative
```

Floor division and modulus behave mathematically for negative numbers, unlike `/` and `%`:

```
-7 / 2;                    // -3     truncates toward zero
Math.floorDiv(-7, 2);      // -4     floors
-7 % 2;                    // -1     sign of the dividend
Math.floorMod(-7, 2);      // 1      always in [0, 2): use it for array and
hash indexes
Math.clamp(15, 0, 10);     // 10     (Java 21) keeps a value within a range
Math.ceilDiv(7, 2);        // 4      (Java 18)
```

- `Math.random()` returns a `double` in `[0, 1)` from a shared generator. For concurrency use `ThreadLocalRandom.current().nextInt(a, b)`; for new code, the `RandomGenerator` interface (Java 17) gives modern algorithms. For security use `SecureRandom`.
- `Math.fma(a, b, c)` computes `a*b+c` with a single rounding; `Math.ulp(x)` is the gap to the next double.
- **`StrictMath`** guarantees bit-for-bit identical results on every platform (the fdlibm algorithms); `Math` may use faster intrinsics whose last digit can differ. Use `StrictMath` only when exact reproducibility matters.

System

```
System.out.println("hi");  System.err.println("error");  // standard output
and error streams
System.in.read();                                        // standard input
(a raw InputStream)
System.setOut(new PrintStream(new FileOutputStream("out.log")));   //
redirect (rarely wise)

long t0 = System.nanoTime();                             // monotonic: use
for ELAPSED time
work();
long ms = (System.nanoTime() - t0) / 1_000_000;
long wall = System.currentTimeMillis();                  // wall clock since
1970: for TIMESTAMPS, can jump backwards

int[] a = {1, 2, 3, 4, 5};
System.arraycopy(a, 0, a, 1, 4);                         // [1, 1, 2, 3, 4]
fast, native, overlap-safe

System.getProperty("java.version");  System.getProperty("user.dir");
System.getProperty("os.name");
System.getenv("HOME");                                   // an environment
variable (null if absent)
System.lineSeparator();                                  // "\n" or "\r\n"
System.identityHashCode(obj);                            // the
Object.hashCode() value, even if hashCode is overridden
System.exit(1);                                          // shuts down the
JVM with an exit status (runs shutdown hooks)
System.gc();                                             // a HINT only; do
not rely on it
System.Logger log = System.getLogger("app");            // Java 9: a minimal
logging facade
```

- Measure elapsed time with **`nanoTime`** (monotonic, not related to the calendar) and timestamps with `currentTimeMillis` or `java.time.Instant`.
- `System.arraycopy` checks bounds and types and throws `IndexOutOfBoundsException` or `ArrayStoreException`; it copies **references** only for object arrays (shallow, 3.3.7).

- Avoid `System.exit` in libraries and web apps; it kills the whole JVM. The Security Manager, once reachable through `System`, is permanently disabled in recent JDKs (Java 24).

Runtime

One `Runtime` object represents the running JVM: `Runtime.getRuntime()`.

```
Runtime rt = Runtime.getRuntime();
rt.availableProcessors();         // CPU count; respects container (cgroup)
limits since Java 10
rt.maxMemory();                   // the heap ceiling (-Xmx)
rt.totalMemory();                 // heap currently reserved from the OS
rt.freeMemory();                  // unused part of that reservation
rt.addShutdownHook(new Thread(() -> flushAndClose()));   // runs on normal
exit, Ctrl+C, SIGTERM
Runtime.version().feature();      // 25 on a Java 25 JVM
```

Shutdown hooks run when the JVM exits normally or on `SIGTERM` / `SIGINT`, but **not** after `kill -9`, `Runtime.halt`, or a crash, so they are for best-effort cleanup, not for correctness. Keep them short and thread-safe.

The exception hierarchy

```
Throwable
 |-- Error                        (serious JVM problems; do not catch)
 |     OutOfMemoryError, StackOverflowError, AssertionError,
NoClassDefFoundError, ExceptionInInitializerError
 |-- Exception
       |-- checked exceptions     (must be caught or declared)
       |     IOException, SQLException, InterruptedException,
ClassNotFoundException, TimeoutException
       |-- RuntimeException       (unchecked)
             NullPointerException, IllegalArgumentException
(NumberFormatException), IllegalStateException,
             ClassCastException, ArithmeticException,
UnsupportedOperationException, ArrayStoreException,
             IndexOutOfBoundsException (ArrayIndexOutOfBoundsException,
StringIndexOutOfBoundsException),
             NegativeArraySizeException, ConcurrentModificationException
(java.util)
```

Checked vs unchecked. A **checked** exception (a subclass of `Exception` that is not a `RuntimeException`) must be handled with `try/catch` or declared with `throws`; it is for recoverable conditions outside the program's control (file missing, network down). An **unchecked** exception (`RuntimeException` and `Error`) need not be declared; it normally signals a **programming bug** (null, bad argument, wrong state) and is fixed in code, not caught.

Exception	Typical cause
`NullPointerException`	Using a `null` reference; Java 15+ messages name the variable ("because `s` is null")
`IllegalArgumentException`	A method argument is invalid; `NumberFormatException` is a subclass
`IllegalStateException`	The object is in the wrong state for this call
`ClassCastException`	A bad downcast
`ArithmeticException`	Integer division by zero, or `*Exact` overflow
`ArrayIndexOutOfBoundsException`	Index outside `0..length-1`
`UnsupportedOperationException`	A read-only collection was modified (`List.of(...).add`)
`ConcurrentModificationException`	A collection changed while being iterated
`StackOverflowError`	Runaway recursion
`OutOfMemoryError`	The heap, Metaspace or native memory is exhausted (Section 1)

Useful `Throwable` methods: `getMessage()`, `getCause()`, `getStackTrace()`, `printStackTrace()`, `addSuppressed` and `getSuppressed()` (try-with-resources records exceptions thrown by `close()` as **suppressed** so the primary exception is not lost), and `fillInStackTrace()`, which is the expensive part of creating an exception (filling the stack), so do not use exceptions for ordinary control flow.

```
try {
    process();
} catch (IOException e) {
    throw new IllegalStateException("failed to process order 42", e);    //
wrap and keep the CAUSE
}
```

Always pass the original exception as the **cause**; a replacement without it destroys the stack trace of the real problem. Full exception-handling design (custom exceptions, try-with-resources, retry, API contracts) is a later module.

Threads

`Thread`, `Runnable`, and `ThreadLocal` are covered in the Multithreading notes. In `java.lang`: `Thread.ofVirtual().start(task)` creates a **virtual thread** (Java 21); `ThreadLocal<T>` gives each thread its own copy of a variable (remove it with `remove()` in pooled threads to avoid leaks); `ScopedValue` (final in Java 25) shares immutable data down a call tree and is the virtual-thread-friendly alternative.

Core interfaces

Interface	Single method (or purpose)	Why it matters
`Comparable<T>`	`compareTo`	Natural ordering (3.5.3)
`CharSequence`	`length`, `charAt`, `subSequence`, `toString`, `isEmpty` (15), `chars()`	Implemented by `String`, `StringBuilder`, `CharBuffer`; lets APIs accept any text
`Iterable<T>`	`iterator()`, default `forEach`, `spliterator`	**Anything `Iterable` works in a for-each loop**
`AutoCloseable`	`close()`	Target of try-with-resources; `Closeable` (java.io) narrows its exception to `IOException`
`Cloneable`	marker	Permits `Object.clone()` (3.3.7)
`Appendable`	`append`	Common to `StringBuilder`, `Writer`
`Runnable`	`run()`	A task for a thread or executor

```
class Range implements Iterable<Integer> {                    // makes Range
usable in for-each
    private final int lo, hi;
    Range(int lo, int hi) { this.lo = lo; this.hi = hi; }
    public Iterator<Integer> iterator() {
        return new Iterator<>() {
            int next = lo;
            public boolean hasNext() { return next < hi; }
            public Integer next() { return next++; }
        };
    }
}
for (int n : new Range(1, 4)) System.out.print(n + " ");     // 1 2 3
```

Process, ProcessBuilder and StackWalker

```
Process p = new ProcessBuilder("git",
"status").redirectErrorStream(true).start();   // preferred way to run
another program
String out = new String(p.getInputStream().readAllBytes(),
StandardCharsets.UTF_8);
int code = p.waitFor();
// Runtime.exec(String) is deprecated (Java 18): it splits the string
unreliably

ProcessHandle.current().pid();                                  // this
JVM's process id (Java 9)
StackWalker.getInstance().walk(s ->
s.limit(3).map(StackWalker.StackFrame::getMethodName).toList());   // cheap,
lazy stack inspection
```

Never build command strings from user input (command injection); pass arguments as separate list elements.

Standard annotations

Annotation	Purpose
`@Override`	Compiler checks that the method really overrides (3.3.4)

Annotation	Purpose
`@Deprecated(since = "21", forRemoval = true)`	Marks an API as discouraged, or scheduled for removal (callers get a stronger warning)
`@SuppressWarnings("unchecked")`	Silences a specific warning; apply to the smallest scope
`@SafeVarargs`	Declares that a generic varargs method is type-safe (final, static or private methods)
`@FunctionalInterface`	Compiler checks for exactly one abstract method (3.4.5)

New in Java 25

Small programs can use the `java.lang.IO` helper class (`IO.println(...)`, `IO.readln("prompt")`) and compact source files, which Section 2.10 mentioned; they are conveniences for scripts and teaching rather than something production code needs.

Interview traps

- *`nanoTime` vs `currentTimeMillis`?* `nanoTime` is a monotonic counter for measuring elapsed time; `currentTimeMillis` is wall-clock time that can change.
- *How does `Math.round(-2.5)` behave?* It returns `-2` (rounds half up toward positive infinity).
- *`Math` vs `StrictMath`?* `Math` may use faster platform-specific implementations; `StrictMath` is exactly reproducible.
- *Checked vs unchecked exceptions?* Checked must be handled or declared; unchecked (`RuntimeException`, `Error`) need not be.
- *Why is `Error` not to be caught?* It signals JVM-level problems (`OutOfMemoryError`) that the program cannot reasonably recover from.
- *What does `System.gc()` do?* It only suggests a collection.
- *Do shutdown hooks always run?* Not on `kill -9`, `halt`, or a JVM crash.
- *What does a class need to be used in a for-each loop?* To implement `Iterable`.
- *Which exception do you get from `List.of(1).add(2)`?* `UnsupportedOperationException`.

3.6.5 Capstone, Part 1: Design Principles

Parts 3.1 to 3.5 taught the **language features** of OOP. Writing good object-oriented code needs **judgment** about how to use them. At SDE-3 level, interviews test that judgment: not "what is an interface" but "when would you introduce one, and why?" The principles below are heuristics that guide those decisions, and each maps back to features you now know.

Cohesion and coupling

Cohesion is how strongly the members of one class belong together. **Coupling** is how much one class depends on others. Aim for **high cohesion and low coupling**.

	High cohesion	Low cohesion
A class	`EmailValidator` validates email addresses	`Utils` with date parsing, email checks, and file copying
Effect	Easy to name, understand, test, reuse	A "god class" that changes for many unrelated reasons

	Low coupling	High coupling
A class depends on	An **interface** (`PaymentGateway`)	A concrete class, its fields, and its internals
Effect	Change one side without touching the other	A change ripples through many classes

Tools you already have: **interfaces** and **abstract classes** reduce coupling to implementations; **private fields and small public APIs** (3.1.3) reduce coupling to internals; **composition** (3.3.2) rather than inheritance (3.3.6) reduces coupling to a parent's implementation.

Law of Demeter ("talk to friends, not strangers"): a method should call only methods of its own object, its parameters, objects it creates, and its direct fields. A chain of calls through other objects shows excess coupling:

```
String city = order.getCustomer().getAddress().getCity();     // "train
wreck": Order's caller knows Customer and Address

String city = order.shippingCity();                            // better:
Order hides its structure and exposes the behavior
```

Tell, don't ask (3.1.3): tell an object what to do instead of pulling its data out and deciding for it.

The SOLID principles

Five principles for class design, with a short example of each.

S: Single Responsibility Principle: a class should have **one reason to change**.

```
// Violation: three reasons to change (pricing rules, report layout,
database)
class Invoice {
    double total() { /* pricing */ return 0; }
    String print() { /* formatting */ return ""; }
    void save()    { /* JDBC */ }
}

// Better: each class has one job
class Invoice            { double total() { return 0; } }              //
domain logic
class InvoicePrinter     { String print(Invoice i) { return ""; } }     //
presentation
class InvoiceRepository  { void save(Invoice i) { } }                   //
persistence
```

O: Open/Closed Principle: software entities should be **open for extension, closed for modification**. Add new behavior by writing new code, not by editing tested code (3.3.5).

```
// Violation: every new discount edits this method
double discount(String type, double total) {
    if (type.equals("FESTIVE")) return total * 0.9;
    else if (type.equals("BULK")) return total - 500;
    return total;
}

// Better: a new discount is a new class (or lambda); this code never changes
interface DiscountPolicy { double apply(double total); }
record Festive() implements DiscountPolicy { public double apply(double t) {
return t * 0.9; } }
record Bulk()    implements DiscountPolicy { public double apply(double t) {
return t - 500; } }
double payable(DiscountPolicy p, double total) { return p.apply(total); }
```

For a **closed** set of variants, a sealed hierarchy with an exhaustive `switch` (3.5.7) is an equally valid answer: the compiler tells you every place to update.

L: Liskov Substitution Principle: subtypes must be usable wherever their parent type is expected, without breaking correctness (3.3.6). Warning signs: an override that throws `UnsupportedOperationException`, or code that checks `instanceof` to treat a subclass specially. `Square extends Rectangle` is the textbook violation.

I: Interface Segregation Principle: clients should not be forced to depend on methods they do not use. Prefer **small, role-focused interfaces** (3.4.4).

```
// Violation: a robot must implement eat()
interface Worker { void work(); void eat(); }

// Better
interface Workable { void work(); }
interface Feedable { void eat(); }
class Human implements Workable, Feedable { public void work() { } public
void eat() { } }
class Robot implements Workable            { public void work() { } }
```

JDK examples: `Comparable`, `Closeable`, `Runnable` are one-method interfaces; a reporting service that takes a `ReadRepository` cannot delete data.

D: Dependency Inversion Principle: high-level modules should depend on **abstractions**, not on concrete low-level classes; details depend on abstractions (3.4.6).

```
// Violation: the service builds and depends on a concrete database class
class OrderService {
    private final MySqlOrderRepository repo = new MySqlOrderRepository();
}

// Better: depend on an interface, receive the implementation (dependency
injection)
class OrderService {
    private final OrderRepository repo;
    OrderService(OrderRepository repo) { this.repo = repo; }      //
constructor injection
}
```

Spring supplies the implementation (`@Service`, `@Repository`, constructor injection), and tests pass a fake. Note the difference between the **principle** (depend on abstractions), **inversion of control** (a framework calls your code and creates objects), and **dependency injection** (the technique that supplies collaborators from outside).

Principle	Main Java tools
Single Responsibility	Small classes, packages, composition
Open/Closed	Interfaces, abstract classes, polymorphism, strategy; sealed types for closed sets
Liskov Substitution	Careful overriding, `final`, contract tests, composition instead of inheritance
Interface Segregation	Small interfaces, `extends` to combine them, default methods sparingly
Dependency Inversion	Interfaces, constructor injection, Spring beans

Other principles worth knowing

Principle	Meaning	Caution
DRY (Don't Repeat Yourself)	Each piece of **knowledge** has one authoritative place	Two similar-looking snippets may represent different knowledge; a wrong abstraction costs more than duplication
KISS	Prefer the simplest design that works	Simple does not mean naive
YAGNI (You Aren't Gonna Need It)	Do not build for imagined future needs	Keep seams that are cheap (interfaces at real boundaries)
Composition over inheritance	Reuse by holding collaborators (3.3.6)	Inheritance is still right for true IS-A
Favor immutability	Immutable values are safe and simple (3.5.4)	Do not force it on entities that must change
Fail fast	Validate arguments in constructors and methods, throw early	Use meaningful exceptions
Least privilege and least surprise	Narrow visibility (3.1.3); behave as callers expect	
Design by contract	Document preconditions, postconditions, invariants	Check them in tests
Separation of concerns	Layers for web, business and data	Too many layers is its own smell

Rule of three: the first time you write something, just write it; the second time, note the duplication; the third time, **refactor** into an abstraction. By then you know what really varies.

Code smells and the OOP fix

Smell	Symptom	Remedy
God class	One class does everything	Split by responsibility (SRP)
Long method	Dozens of lines, many levels of nesting	Extract methods; name the steps
Switch on type	`if (x instanceof A) ... else if (x instanceof B)` repeated	Polymorphism; or a sealed type with a pattern `switch`
Primitive obsession	`String email`, `int amountPaise` everywhere	Value objects and records (`Email`, `Money`)
Feature envy	A method uses another class's data more than its own	Move the method to the class that owns the data
Anemic domain model	Entities are bags of getters and setters; all logic in services	Move behavior into the domain objects
Data clumps	The same 3 parameters travel together	Introduce a parameter object (record)
Shotgun surgery	One change edits many classes	Gather the responsibility in one place
Inheritance for reuse	A subclass only wants some parent methods	Composition and delegation
Mutable shared state	Static maps, public fields	Encapsulate, make immutable, inject

Where patterns fit

Design patterns are named solutions to recurring design problems. Most are built from the features in this track.

Pattern	Idea	Java and Spring examples
Strategy	Interchangeable algorithms behind an interface	`Comparator`, `DiscountPolicy`

Pattern	Idea	Java and Spring examples
Factory / static factory	Hide which class is created	`List.of`, `Calendar.getInstance`
Builder	Assemble complex immutable objects step by step	`StringBuilder`, `Pizza.Builder`, `HttpRequest.newBuilder()`
Singleton	One instance	Enum singleton; a Spring bean (default scope)
Decorator	Wrap an object to add behavior	`BufferedReader` around `Reader`
Adapter	Make one interface fit another	`Arrays.asList`, `InputStreamReader`
Observer / listener	Notify subscribers of events	`ApplicationEvent`, `PropertyChangeListener`
Template Method	Fixed algorithm with overridable steps (3.3.6)	`JdbcTemplate`, `HttpServlet`
Proxy	A stand-in that controls access	Spring AOP and `@Transactional` (CGLIB or JDK proxies)
Command	A request as an object	`Runnable`, `Callable`
State	Behavior depends on current state	An enum with behavior (3.5.6)
Visitor / pattern matching	Add operations over a closed hierarchy	Sealed types plus `switch` (3.5.7)

A full study of the Gang of Four patterns is a later module; this table shows that you already have the building blocks.

How it looks in a Spring Boot service

```
Controller     (translates HTTP to calls; thin)            depends on the
Service interface
   |
Service        (business rules; transactions)              depends on
Repository and Gateway interfaces
   |
Repository     (persistence)  and  Gateway (external APIs)  implement
interfaces; hide JPA and HTTP details
   |
Domain model   (entities, value objects, records, enums, sealed results)
```

- **SRP:** each layer has one job. **DIP:** upper layers depend on interfaces. **OCP:** new payment providers are new beans. **ISP:** separate read and write repositories where useful.
- **Immutability:** DTOs and value objects are records; entities are mutable but encapsulated.
- **Fail fast:** constructors and service methods validate inputs.

Trade-offs, not rules

Principles can be over-applied. An interface with one implementation, a factory for a class that never varies, and ten tiny classes where one clear one would do all add cost without benefit. A good design is **as simple as the problem allows, with seams where change is likely**. In an interview, state the principle, then say **what the change request would be** that makes it worth applying.

Interview traps and talking points

- *Explain SOLID with an example.* Pick one small domain (payments or notifications) and show a violation and a fix for each letter, as above.
- *High cohesion and low coupling: what do they mean and how do you achieve them?* Focused classes; depend on interfaces; hide internals.
- *Dependency Inversion vs dependency injection vs inversion of control?* A design principle, a technique, and the framework's control flow.
- *When is inheritance better than composition?* A real IS-A that honors Liskov, maintained together, with polymorphic use.

- *How do you spot a god class?* Many unrelated fields and methods, many imports, many reasons to change, and tests that need lots of setup.
- *DRY or duplication?* Remove duplication of **knowledge**; tolerate incidental similarity; wait for the third occurrence.

3.6.6 Capstone, Part 2: Worked Design Exercises

These exercises imitate the **low-level design (LLD)** rounds common in SDE-3 interviews. For each one: read the requirements, **design on paper first**, then compare with the model design, and finally apply the "change requests" to test whether your design bends or breaks.

A method for design questions

1. **Clarify** the requirements and ask about scope (concurrency? persistence? what is out of scope?).
2. **Find the nouns** (candidate classes) and the **verbs** (candidate methods).
3. For each noun decide: **entity** (has identity and changes), **value object** (equal by value, immutable: a record), **enum** (fixed constants), or **service** (stateless behavior).
4. Find **what varies** and hide it behind an interface (strategy); find **closed sets** and model them as an enum or a sealed type.
5. Decide **relationships** (IS-A, HAS-A: composition or aggregation) and **who owns each invariant**.
6. Sketch the **public API**, then the **main flow**.
7. Think about **concurrency, failure and persistence**.
8. State the **trade-offs** and the **extension points**.

Exercise 1: Order checkout with payments and discounts

Requirements. A customer places an order with several lines (sku, quantity, unit price). A discount policy may reduce the total. The customer pays by UPI, card or wallet; a payment can be approved, declined or pending. An order moves through NEW, PAID, SHIPPED, DELIVERED or CANCELLED, with only certain transitions allowed. The business will add more payment methods and discount rules.

Model.

```
Money (record)  OrderLine (record)  OrderStatus (enum, transition rules)
Order (entity, aggregate root: HAS-A lines, owns the status invariants)
PaymentMethod (interface)  <- UpiPayment, CardPayment, WalletPayment
strategy
PaymentResult (sealed interface) = Approved | Declined | Pending
closed set of outcomes
DiscountPolicy (interface) <- NoDiscount, FestiveDiscount, BulkDiscount
strategy
OrderRepository (interface)
dependency inversion
CheckoutService (depends only on the interfaces)
```

Key code.

```
public record Money(long paise) {
    public static final Money ZERO = new Money(0);
    public Money { if (paise < 0) throw new
IllegalArgumentException("negative money"); }
    public Money plus(Money o)  { return new Money(Math.addExact(paise,
o.paise)); }
    public Money times(int n)   { return new Money(Math.multiplyExact(paise,
(long) n)); }
}

public record OrderLine(String sku, int qty, Money unitPrice) {
    public OrderLine {
        Objects.requireNonNull(sku); Objects.requireNonNull(unitPrice);
        if (qty <= 0) throw new IllegalArgumentException("qty must be
positive");
    }
    public Money total() { return unitPrice.times(qty); }
}

public enum OrderStatus {
    NEW, PAID, SHIPPED, DELIVERED, CANCELLED;
    public boolean canMoveTo(OrderStatus next) {
        return switch (this) {                                    //
exhaustive: a new status forces a decision here
            case NEW       -> next == PAID || next == CANCELLED;
            case PAID      -> next == SHIPPED || next == CANCELLED;
            case SHIPPED   -> next == DELIVERED;
            case DELIVERED, CANCELLED -> false;
        };
```

```
    }
}

public final class Order {                                          // the
entity guards its own invariants
    private final String id;
    private final List<OrderLine> lines = new ArrayList<>();
    private OrderStatus status = OrderStatus.NEW;

    public Order(String id) { this.id = Objects.requireNonNull(id); }
    public void addLine(OrderLine line) {
        if (status != OrderStatus.NEW) throw new IllegalStateException("order
is " + status);
        lines.add(line);
    }
    public Money total()            { return
lines.stream().map(OrderLine::total).reduce(Money.ZERO, Money::plus); }
    public List<OrderLine> lines()  { return List.copyOf(lines); }       //
never expose the mutable list
    public OrderStatus status()     { return status; }
    public String id()              { return id; }
    public void markPaid()          { moveTo(OrderStatus.PAID); }
    public void cancel()            { moveTo(OrderStatus.CANCELLED); }
    private void moveTo(OrderStatus next) {
        if (!status.canMoveTo(next)) throw new IllegalStateException(status +
" -> " + next + " not allowed");
        status = next;
    }
}

public interface PaymentMethod   { PaymentResult pay(Money amount); }
public interface DiscountPolicy  { Money apply(Money total); }
public interface OrderRepository { void save(Order order); Optional<Order>
findById(String id); }

public sealed interface PaymentResult {
    record Approved(String txnId)   implements PaymentResult { }
    record Declined(String reason)  implements PaymentResult { }
    record Pending(String ref)      implements PaymentResult { }
}

public class CheckoutService {
    private final DiscountPolicy discount;
```

```
    private final OrderRepository repo;
    public CheckoutService(DiscountPolicy discount, OrderRepository repo) {
// constructor injection
        this.discount = discount;
        this.repo = repo;
    }
    public PaymentResult checkout(Order order, PaymentMethod method) {
        Money payable = discount.apply(order.total());
        PaymentResult result = method.pay(payable);
        switch (result) {
// exhaustive over the sealed type
            case PaymentResult.Approved a  -> { order.markPaid();
repo.save(order); }
            case PaymentResult.Pending p   -> repo.save(order);
            case PaymentResult.Declined d  -> { /* leave the order NEW so the
customer can retry */ }
        }
        return result;
    }
}
```

Why this design works

- **Encapsulation and invariants:** `Order` owns its status and its lines; nothing outside can put it into an illegal state (3.1.3).
- **Immutable value types:** `Money` and `OrderLine` are records with validation (3.5.5). `Money` uses `long` paise and `*Exact` arithmetic.
- **Strategy and OCP:** a new payment method or discount is a new class; `CheckoutService` does not change (3.3.5, 3.6.5).
- **Sealed result:** the compiler forces `checkout` to handle every outcome (3.5.7).
- **DIP:** `CheckoutService` depends on interfaces; tests inject fakes, Spring injects real beans.
- **Enum state machine:** transition rules live in one place and are exhaustive.

Change requests to test it

Change	Effect on the design
Add `NetBankingPayment`	One new class implementing `PaymentMethod`; nothing else changes

Change	Effect on the design
Add a coupon discount	One new `DiscountPolicy`
Add the status `RETURNED`	`canMoveTo` fails to compile until you decide the rules (a feature)
Add partial payments	New `PaymentResult.PartiallyApproved`; the compiler flags every switch to update
Persist in PostgreSQL	An `OrderRepository` implementation; the domain is untouched
Two threads pay the same order	Not handled yet: add optimistic locking (a version field) or a lock around the status change

Exercise 2: A notification system

Requirements. Send notifications (order shipped, password reset) through email, SMS or push, according to each user's preferences. Sending can fail temporarily and should be retried; some channels must be rate limited. New channels will be added.

Model.

```
Notification (record)   Channel (enum)   UserPreferences (record: which
channels, per user)
NotificationChannel (interface)  <- EmailChannel, SmsChannel, PushChannel
AbstractChannel (skeleton: template method for validate -> send -> audit)
RetryingChannel, RateLimitedChannel   (decorators: implement the same
interface and wrap another)
NotificationService (picks the channels for a user and sends)
```

Key code (the decorators are the new idea).

```
public record Notification(String userId, String subject, String body) {
    public Notification { Objects.requireNonNull(userId);
Objects.requireNonNull(body); }
}
public enum Channel { EMAIL, SMS, PUSH }

public interface NotificationChannel {
    Channel channel();
    void send(Notification n);
}
```

```
public final class RetryingChannel implements NotificationChannel {        //
Decorator: composition, not inheritance
    private final NotificationChannel delegate;
    private final int attempts;
    public RetryingChannel(NotificationChannel delegate, int attempts) {
        if (attempts < 1) throw new IllegalArgumentException("attempts >=
1");
        this.delegate = Objects.requireNonNull(delegate);
        this.attempts = attempts;
    }
    @Override public Channel channel() { return delegate.channel(); }
    @Override public void send(Notification n) {
        RuntimeException last = null;
        for (int i = 0; i < attempts; i++) {
            try { delegate.send(n); return; }
            catch (RuntimeException e) { last = e; }
// a real version adds backoff
        }
        throw last;
    }
}

public class NotificationService {
    private final Map<Channel, NotificationChannel> channels;
    public NotificationService(List<NotificationChannel> all) {
// Spring injects every bean of this type
        this.channels =
all.stream().collect(Collectors.toMap(NotificationChannel::channel,
Function.identity()));
    }
    public void notify(Notification n, Set<Channel> preferred) {
        for (Channel c : preferred) {
            NotificationChannel ch = channels.get(c);
            if (ch != null) ch.send(n);
        }
    }
}

// Wiring (for example in a configuration class): behavior is composed, not
inherited
NotificationChannel sms = new RetryingChannel(new RateLimitedChannel(new
SmsChannel(), 10), 3);
```

Discussion points

- **ISP and DIP:** the service sees only `NotificationChannel`.
- **Decorators vs inheritance:** `RetryingChannel(RateLimitedChannel(SmsChannel))` composes retry and rate limit freely; with inheritance you would need a class per combination (3.3.6).
- **Template method** (3.4.6): an `AbstractChannel` with a `final send` that validates, calls `doSend`, and audits keeps common steps in one place.
- **Decoupling from the caller:** publish a domain event (`OrderShipped`) and let a listener call `NotificationService`, so the order code does not know about notifications (Observer).
- **Failure:** retries are for transient errors only; sending can be moved to a queue (at-least-once delivery, so make notifications idempotent).

Change requests: add `WhatsAppChannel` (one new class, one new `Channel` constant); fall back to SMS if push fails (a `FallbackChannel` decorator); templates per language (a `TemplateRenderer` collaborator).

Exercise 3: A parking lot

Requirements. A lot has spots of three sizes. Vehicles (bike, car, truck) enter, get a ticket and a spot that fits, and pay on exit by duration and vehicle type. Several gates work **at the same time**. Pricing and spot-selection rules will change.

Model.

```
VehicleType (enum)   SpotType (enum, knows what fits)   Vehicle (record)
Ticket (record)
ParkingSpot (entity: thread-safe occupancy)
SpotAllocationStrategy (interface: rank candidate spots)   PricingStrategy
(interface)
ParkingLot (aggregate: HAS-A spots; coordinates enter and exit; uses a Clock)
```

Key code.

```
enum VehicleType { BIKE, CAR, TRUCK }
enum SpotType {
    SMALL, MEDIUM, LARGE;
    boolean fits(VehicleType v) {
        return switch (this) {
            case SMALL  -> v == VehicleType.BIKE;
```

```
            case MEDIUM -> v == VehicleType.BIKE || v == VehicleType.CAR;
            case LARGE  -> true;
        };
    }
}
record Vehicle(String plate, VehicleType type) { }
record Ticket(String id, Vehicle vehicle, String spotId, Instant entry) { }

final class ParkingSpot {
    private final String id;
    private final SpotType type;
    private Vehicle parked;                                          // guarded
by 'this'
    ParkingSpot(String id, SpotType type) { this.id = id; this.type = type; }
    String id() { return id; }
    SpotType type() { return type; }
    synchronized boolean tryPark(Vehicle v) {                        // check-
and-set must be ONE atomic step
        if (parked != null || !type.fits(v.type())) return false;
        parked = v;
        return true;
    }
    synchronized void free() { parked = null; }
    synchronized boolean isFree() { return parked == null; }
}

interface SpotAllocationStrategy { List<ParkingSpot> rank(List<ParkingSpot>
spots, Vehicle v); }   // best candidate first
interface PricingStrategy       { long price(Vehicle v, Duration stay); }
// in paise

final class ParkingLot {
    private final List<ParkingSpot> spots;
    private final SpotAllocationStrategy allocation;
    private final PricingStrategy pricing;
    private final Clock clock;                                        //
injected, so tests control time
    private final Map<String, Ticket> active = new ConcurrentHashMap<>();
    private final Map<String, ParkingSpot> spotsById;

    ParkingLot(List<ParkingSpot> spots, SpotAllocationStrategy a,
PricingStrategy p, Clock clock) {
        this.spots = List.copyOf(spots);
```

```
        this.allocation = a; this.pricing = p; this.clock = clock;
        this.spotsById =
spots.stream().collect(Collectors.toMap(ParkingSpot::id,
Function.identity()));
    }

    Optional<Ticket> enter(Vehicle v) {
        for (ParkingSpot s : allocation.rank(spots, v)) {         // try
candidates in preference order
            if (s.tryPark(v)) {                                   // atomic
per spot: two gates cannot take the same spot
                Ticket t = new Ticket(UUID.randomUUID().toString(), v,
s.id(), clock.instant());
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();                                  // lot full
for this vehicle
    }

    long exit(String ticketId) {
        Ticket t = active.remove(ticketId);                       // atomic:
a ticket can be used once
        if (t == null) throw new IllegalArgumentException("unknown or used
ticket");
        spotsById.get(t.spotId()).free();
        return pricing.price(t.vehicle(), Duration.between(t.entry(),
clock.instant()));
    }
}
```

Why this design works

- **Enums with behavior** (`SpotType.fits`) keep size rules in one place; **records** make `Vehicle` and `Ticket` immutable values.
- **Strategies** isolate the two rules that will change (where to park, how to price) behind interfaces.
- **Concurrency is designed in:** the only contended operation, taking a spot, is a single synchronized method per spot (fine-grained locking), so two gates scale; the ticket map is a `ConcurrentHashMap` and `remove` is atomic.

- **Testability:** an injected `Clock` and strategies allow deterministic tests without sleeping.
- **Encapsulation:** nobody outside touches `parked`.

Change requests: add EV-charging spots (a new `SpotType` constant and a strategy that prefers them for EVs); peak-hour pricing (a new `PricingStrategy`); a display board of free spots (an observer notified on `tryPark` and `free`); reserve spots in advance (a new reservation state, which suggests promoting the spot's state to a small state machine); persistence and several lots (an interface for storage and a `ParkingLotRepository`).

Exercise 4: Shapes, twice

Implement the same problem two ways and decide which fits: compute `area()` and `perimeter()` for circles, rectangles and triangles.

```
// Version A: open hierarchy with virtual dispatch
interface Shape { double area(); double perimeter(); }
record Circle(double r)               implements Shape { public double area()
{ return Math.PI * r * r; } public double perimeter() { return 2 * Math.PI *
r; } }
record Rectangle(double w, double h)  implements Shape { public double area()
{ return w * h; } public double perimeter() { return 2 * (w + h); } }

// Version B: closed hierarchy; operations live outside
sealed interface Shape2 permits Circle2, Rectangle2 { }
record Circle2(double r) implements Shape2 { }
record Rectangle2(double w, double h) implements Shape2 { }
static double area(Shape2 s) { return switch (s) { case Circle2 c -> Math.PI
* c.r() * c.r(); case Rectangle2 r -> r.w() * r.h(); }; }
```

Decide with the change you expect. If **new shape types** keep arriving and the operations are stable, Version A is easier (the new class adds all its operations). If **new operations** keep arriving (`draw`, `toSvg`, `scale`) and the set of shapes is stable and owned by you, Version B is easier, and the compiler flags every function that misses a new shape. That is the expression problem (3.5.7), and naming it is a strong interview answer.

What to say in the interview

- State assumptions and the **core entities** before coding.
- Name each design decision with its **reason** ("I made `Money` a record because it is a value and must be immutable").

- Point to the **extension points** and the **change request** each one absorbs.
- Mention **concurrency and failure** even if out of scope, with one concrete mechanism.
- Keep it **small first**; add patterns only when a requirement demands them.

Review Questions: Part 3.6 and the Whole OOP Track

Attempt every question first. Part A is multiple choice, Part B is short answer, Part C is coding and prediction, and Part D is senior level and cuts across all of Part 3 (3.1 to 3.6). All answers and explanations are consolidated in the next section.

Part A: Multiple Choice

Q1. What is printed?

```
String a = "java";
String b = new String("java");
System.out.println((a == b) + " " + a.equals(b));
```

- (a) `true true`
- (b) `false true`
- (c) `false false`
- (d) `true false`

Q2. What is printed?

```
System.out.println(("ja" + "va") == "java");
```

- (a) `true`
- (b) `false`
- (c) Compile error
- (d) Depends on the JVM

Q3. What is printed?

```
String p = "ja";
System.out.println((p + "va") == "java");
```

- (a) `true`
- (b) `false`

- (c) Compile error
- (d) `null`

Q4. What does `"Aa".hashCode() == "BB".hashCode()` evaluate to?

- (a) `false`, because the strings differ
- (b) `true`: both are 2112, a hash collision
- (c) Compile error
- (d) It is random

Q5. Since Java 7, the string pool is located:

- (a) In PermGen
- (b) On the ordinary heap
- (c) On the thread stack
- (d) In native memory outside the JVM

Q6. Since Java 9, a `String` stores its characters in:

- (a) A `char[]` always
- (b) A `byte[]` plus a coder flag (Latin-1 or UTF-16)
- (c) An `int[]` of code points
- (d) A linked list of characters

Q7. What happens?

```
String s = String.valueOf(null);
```

- (a) `s` is `"null"`
- (b) `s` is `null`
- (c) `NullPointerException`
- (d) Compile error

Q8. What is the length of the resulting array?

```
int n = "a.b".split(".").length;
```

- (a) 2
- (b) 3
- (c) 0

- (d) 1

Q9. What is the default capacity of `new StringBuilder()`?

- (a) 0
- (b) 10
- (c) 16
- (d) 32

Q10. What does `new StringBuilder("abc").capacity()` return?

- (a) 3
- (b) 16
- (c) 19
- (d) 32

Q11. What is printed?

```
StringBuilder a = new StringBuilder("x");
StringBuilder b = new StringBuilder("x");
System.out.println(a.equals(b) + " " + a.toString().equals(b.toString()));
```

- (a) `true true`
- (b) `false true`
- (c) `false false`
- (d) `true false`

Q12. What does this append?

```
StringBuilder sb = new StringBuilder();
sb.append('a' + 1);
```

- (a) `b`
- (b) `98`
- (c) `a1`
- (d) Nothing; compile error

Q13. Which of these has synchronized methods?

- (a) `String`
- (b) `StringBuilder`

- (c) `StringBuffer`
- (d) All three

Q14. What is printed?

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d));
```

- (a) `true true`
- (b) `true false`
- (c) `false false`
- (d) `false true`

Q15. What is printed?

```
Long l = 5L;
System.out.println(l.equals(5));
```

- (a) `true`
- (b) `false`
- (c) Compile error
- (d) `ClassCastException`

Q16. What happens?

```
Integer a = null;
boolean flag = true;
Integer r = flag ? a : 0;
```

- (a) `r` is `null`
- (b) `r` is `0`
- (c) `NullPointerException`
- (d) Compile error

Q17. What is printed?

```
List<Integer> l = new ArrayList<>(List.of(10, 20, 30));
l.remove(1);
System.out.println(l);
```

- (a) `[20, 30]`

- (b) `[10, 30]`
- (c) `[10, 20, 30]`
- (d) `[10, 20]`

Q18. Which wrapper class has no cache for `valueOf`?

- (a) `Integer`
- (b) `Boolean`
- (c) `Double`
- (d) `Character`

Q19. What is printed?

```
System.out.println(Double.valueOf(0.0).equals(-0.0));
```

- (a) `true`
- (b) `false`
- (c) Compile error
- (d) `NaN`

Q20. What does `Integer.parseInt("12a")` do?

- (a) Returns 12
- (b) Returns 0
- (c) Throws `NumberFormatException`
- (d) Returns `null`

Q21. What is printed?

```
System.out.println(Math.round(-2.5) + " " + Math.abs(Integer.MIN_VALUE));
```

- (a) `-3 2147483648`
- (b) `-2 -2147483648`
- (c) `-2 2147483648`
- (d) `-3 -2147483648`

Q22. What is `Math.floorMod(-7, 2)`?

- (a) -1
- (b) 1

- (c) 0
- (d) -3

Q23. Which call is the right tool to measure elapsed time of a code block?

- (a) `System.currentTimeMillis()`
- (b) `System.nanoTime()`
- (c) `new Date().getTime()`
- (d) `Runtime.totalMemory()`

Q24. Which of these is a checked exception?

- (a) `NullPointerException`
- (b) `IllegalArgumentException`
- (c) `IOException`
- (d) `StackOverflowError`

Q25. Which class most clearly violates the Single Responsibility Principle?

- (a) `Invoice` with methods `total()`, `printPdf()` and `saveToDatabase()`
- (b) `EmailValidator` with `isValid(String)`
- (c) `Money` with `plus` and `minus`
- (d) `InvoiceRepository` with `save` and `findById`

Q26. Which snippet follows the Dependency Inversion Principle?

- (a) `private final MySqlRepo repo = new MySqlRepo();`
- (b) `OrderService(OrderRepository repo) { this.repo = repo; }` where `OrderRepository` is an interface
- (c) `static MySqlRepo REPO = new MySqlRepo();`
- (d) `new MySqlRepo().save(order);` inside every method

Q27. Which is a typical sign of a Liskov Substitution violation?

- (a) A subclass adds a new method
- (b) A subclass overrides a method to throw `UnsupportedOperationException`
- (c) A subclass overrides `toString`
- (d) A subclass is in another package

Q28. `BufferedReader` wrapping a `Reader` to add buffering is an example of which pattern?

- (a) Singleton
- (b) Decorator
- (c) Factory
- (d) Observer

Q29. Which expression is a Law of Demeter violation?

- (a) `order.total()`
- (b) `order.getCustomer().getAddress().getCity()`
- (c) `this.repo.save(order)`
- (d) `new Money(5)`

Q30. A JVM shutdown hook will NOT run when:

- (a) `System.exit(0)` is called
- (b) The process receives `SIGTERM`
- (c) The process is killed with `kill -9`
- (d) `main` returns normally

Part B: Short Answer

Q31. Describe how a `String` is stored in Java 9 and later (compact strings), and what `substring` does.

Q32. Explain the string pool, `intern()`, and the effect of `new String("x")`. When is `intern()` worth using?

Q33. Why is `String.hashCode()` cached, how is it computed, and how can two different strings share a hash?

Q34. Why is concatenation in a loop slow, what does `javac` generate for `+` in Java 8 versus Java 9 and later, and what do you use instead?

Q35. Explain the internals of `StringBuilder` (capacity, growth), and why `StringBuffer`'s synchronization often does not give thread safety.

Q36. Explain autoboxing, the wrapper caches, and four traps (`==`, `equals` across types, unboxing `null`, `List.remove`).

Q37. Discuss the cost of wrappers versus primitives and when to avoid boxing.

Q38. Describe the `*Exact` methods, `floorMod`, `Math.round` semantics, `StrictMath`, and the difference between `nanoTime` and `currentTimeMillis`.

Q39. Draw the `Throwable` hierarchy and explain checked versus unchecked exceptions, the cause chain, and suppressed exceptions.

Q40. What do `Iterable`, `AutoCloseable` and `CharSequence` enable? How does a for-each loop work on an `Iterable`?

Q41. State each SOLID principle in one sentence and name one Java feature that supports it.

Q42. Define cohesion and coupling, the Law of Demeter and "tell, don't ask", with a small example.

Part C: Coding and Prediction

C1. Write `static String reverseWords(String s)` that reverses the order of words, collapses repeated spaces, and uses a `StringBuilder`.

C2. Write `static String compress(String s)` that turns `"aaabbc"` into `"a3b2c1"` using a `StringBuilder`.

C3. Count word frequencies in a text using a `Map<String, Integer>` and `merge`, then print them sorted by count descending and word ascending.

C4. Write `static OptionalInt parseInt(String s)` that returns empty for `null`, blank, non-numeric or out-of-range input and otherwise the value.

C5. Refactor this class using SRP and DIP; show the new classes and interfaces:

```
class ReportService {
    void generateAndSend(List<Sale> sales, String email) {
        StringBuilder sb = new StringBuilder();
        for (Sale s : sales)
sb.append(s.item()).append(',').append(s.amount()).append('\n');
        try (FileWriter w = new FileWriter("report.csv")) {
w.write(sb.toString()); } catch (IOException e) { }
        new SmtpMailer().send(email, "Report", sb.toString());
    }
}
```

C6. Write a decorator `TimedPaymentMethod` that wraps a `PaymentMethod` (from Exercise 1), measures the call with `nanoTime`, logs the duration, and rethrows any exception wrapped with its cause.

C7. Add a `PeakHourPricing` decorator to the parking lot (Exercise 3) that doubles the price of any stay that starts between 17:00 and 20:00, wrapping any other `PricingStrategy`.

C8. Predict the output:

```
String s1 = "hel" + "lo";
String s2 = "hello";
String s3 = new String("hello");
String s4 = s3.intern();
Integer i1 = 100, i2 = 100, i3 = 1000, i4 = 1000;
StringBuilder sb = new StringBuilder("ab");
sb.append(1).append('c').append(2.5).reverse();
System.out.println((s1 == s2) + " " + (s2 == s3) + " " + (s2 == s4) + " " +
(i1 == i2) + " " + (i3 == i4));
System.out.println(sb);
```

Part D: Senior Level and the Whole Track

S1. A service parses 1 million log lines per minute into strings, splits them, and keeps many of them in memory. Discuss memory (compact strings, deduplication, interning, substring copies) and CPU (regex, `split`, builders, allocation) concerns, and what you would measure.

S2. Why does immutability of `String` matter for security? Discuss time-of-check/time-of-use, class loading and file paths, passwords as `char[]`, heap dumps, and the risks of `intern` on untrusted input.

S3. Explain how boxing is implemented (`valueOf`, caches, `-XX:AutoBoxCacheMax`), when the JIT can eliminate boxes, and how you would design numeric-heavy code that avoids boxing. What is Project Valhalla trying to change?

S4. Design an exception strategy for a layered Spring service: checked versus unchecked, wrapping with causes, translation at boundaries, what to log and where, `try-with-resources`, interrupts, and what never to do.

S5. Trace the complete lifetime of `Order o = new Order("A1")`, from class loading and static initialization through allocation, the constructor chain and field initializers, safe publication to another thread, and garbage collection. Name the parts of 3.1 and 3.2 involved at each step.

S6. Predict the output of all four lines and explain each result:

```
class A {
    static String id() { return "A"; }
    String name = "A.name";
    String who() { return "A.who"; }
    String show(Object o) { return "A.show(Object)"; }
    String show(String s) { return "A.show(String)"; }
    String call() { return id() + "/" + who() + "/" + name; }
}
class B extends A {
    static String id() { return "B"; }
    String name = "B.name";
    @Override String who() { return "B.who"; }
    @Override String show(Object o) { return "B.show(Object)"; }
    String show(Integer i) { return "B.show(Integer)"; }
}
A a = new B();  B b = new B();  Object o = "text";
System.out.println(a.call());
System.out.println(a.name + " " + b.name + " " + ((A) b).name);
System.out.println(a.show("x") + " " + a.show(o) + " " + a.show(5));
System.out.println(b.show("x") + " " + b.show(5));
```

S7. Review this design and list every problem, grouped by principle, with the fix for each:

```
public class OrderManager {
    public static Map<String, Order> orders = new HashMap<>();
    public Order createOrder(String name, String email, String address,
List<String> items, double[] prices, String paymentType) {
        Order o = new Order();
        o.id = UUID.randomUUID().toString();
        o.customerName = name;
        o.items = items;
        double total = 0;
        for (double p : prices) total += p;
        o.total = total;
        if (paymentType.equals("UPI")) { /* call UPI */ }
        else if (paymentType.equals("CARD")) { /* call card gateway */ }
        sendEmail(email, "Order " + o.id);
        orders.put(o.id, o);
        saveToDb(o);
        return o;
    }
}
```

S8. A pricing engine must support a fixed set of rule kinds with different parameters, plus partner-defined custom rules loaded at startup. Choose between inheritance, composition, interfaces, enums and sealed types, and justify a combined design.

S9. Design a thread-safe, read-mostly configuration holder for a service. Compare locking, `volatile` with immutable snapshots, and `AtomicReference`; explain which Java features (final fields, records, `List.copyOf`) make the snapshot approach safe.

S10. Model a `Version` (major, minor, patch), a `Currency`, and an `OrderStatus` in a domain model. For each, choose class, record or enum, and explain `equals`, `hashCode`, `compareTo`, immutability and persistence choices.

S11. Build a small service with plain Java constructor injection and no framework: show the composition root, explain what Spring adds, and how you would test it with fakes.

S12. Evaluate each slogan and give its limits: "always program to interfaces", "prefer composition over inheritance", "use records for everything", "use enums instead of constants", "make everything immutable".

Answers: Part 3.6 and the Whole OOP Track

Part A: Multiple Choice

Q	Answer	Explanation
1	(b) `false true`	`new String` is a distinct object; the characters are equal.
2	(a)	A constant expression is folded to `"java"` at compile time, so it is the pooled instance.
3	(b)	`p + "va"` is computed at run time into a new object.
4	(b)	Both hash to 2112: a collision, which is legal.
5	(b)	The pool's strings live on the heap since Java 7.
6	(b)	Java 9's compact strings: `byte[]` plus a `coder`.
7	(c)	`valueOf(null)` binds to `valueOf(char[])` and throws. `String.valueOf((Object) null)` would return `"null"`.

Q	Answer	Explanation
8	(c) 0	`"."` is a regex matching any character, every token is empty, and trailing empty strings are removed. Use `split("\\.")`.
9	(c)	Default capacity is 16.
10	(c)	Initial string length plus 16: 3 + 16 = 19.
11	(b) `false true`	Builders use identity `equals`; compare `toString()` results.
12	(b) `98`	`'a' + 1` is an `int`, so `append(int)` is chosen.
13	(c)	`StringBuffer` methods are synchronized.
14	(b) `true false`	Integer cache covers -128 to 127.
15	(b)	`5` is boxed to `Integer`, and `Long.equals(Integer)` is `false`.
16	(c)	One branch is `Integer`, the other `int`, so `a` is unboxed and `null` throws.
17	(b) `[10, 30]`	`remove(1)` picks `remove(int index)`.
18	(c)	`Double` and `Float` have no cache.
19	(b)	`Double.equals` compares bit patterns, and `0.0` and `-0.0` differ.
20	(c)	`NumberFormatException`.
21	(b)	`Math.round(-2.5)` is `-2`; `Math.abs(Integer.MIN_VALUE)` stays `-2147483648`.
22	(b)	`floorMod` takes the sign of the divisor, giving 1.
23	(b)	`nanoTime` is monotonic and meant for elapsed time.
24	(c)	`IOException` is checked; the others are unchecked or errors.
25	(a)	It mixes domain logic, presentation and persistence.

Q	Answer	Explanation
26	(b)	Depends on an abstraction and receives it from outside.
27	(b)	An override that refuses the inherited contract breaks substitutability.
28	(b)	A wrapper that adds behavior while keeping the same interface.
29	(b)	A chain through `Customer` and `Address` couples the caller to their structure.
30	(c)	`kill -9` gives the JVM no chance to run hooks.

Part B: Short Answer (model answers)

Q31. Since Java 9, a `String` holds a `byte[] value` and a `coder`: Latin-1 text takes one byte per character, other text two bytes (UTF-16). Before Java 9 it was always a `char[]`. The object also caches its `hash`. `substring` copies the characters (since Java 7u6) instead of sharing the parent's array, which avoids retaining a huge array for a small piece.

Q32. The string pool is a table of canonical `String` objects; identical literals and constant expressions refer to one instance. `intern()` returns the pooled instance, adding the string if it is absent. `new String("x")` creates a second `String` object that shares the original's byte array. Since Java 7 the pooled strings are ordinary heap objects, so they can be collected. Use `intern()` only for large numbers of long-lived duplicates with a bounded vocabulary; never on unbounded input. String deduplication in G1, ZGC or Shenandoah is another way to share the byte arrays.

Q33. The hash is `s[0]*31^(n-1) + ... + s[n-1]`, computed on first use and cached in `hash` (with a flag for strings whose hash is 0). Caching is safe because strings are immutable, and it makes strings fast hash keys. Different strings can share a hash (`"Aa"` and `"BB"` are both 2112), which is a collision that `equals` resolves; `HashMap` converts long collision chains into trees to bound the cost.

Q34. Each `s += x` creates a new string and copies all earlier text, so a loop is O(n^2). Java 8 compiled `+` to a `StringBuilder` chain; Java 9 and later emit one `invokedynamic` call to `StringConcatFactory`, which sizes the result exactly. Neither fixes repeated concatenation across iterations, so use one `StringBuilder` outside the loop (or `String.join`, `Collectors.joining`).

Q35. A builder holds a `byte[]` buffer, a coder and a count. The default capacity is 16; when full, the buffer grows to `(old << 1) + 2` and the contents are copied, so appends are amortized O(1). `StringBuffer` synchronizes each method, so individual calls are atomic, but a **sequence** of calls from several threads can still interleave, so it does not make compound operations thread-safe. Use a local `StringBuilder`, or confine and combine, or lock the whole sequence.

Q36. The compiler boxes with `valueOf` and unboxes with `xxxValue()`. `Integer.valueOf` returns cached objects for -128 to 127 (also `Byte`, `Short`, `Long`; `Character` 0 to 127; `Boolean` constants; no cache for `Float` and `Double`). Traps: `==` on boxed values compares identity (true only inside the cache); `Long.equals(Integer)` is false; unboxing `null` throws `NullPointerException` (including through a mixed `Integer` / `int` conditional); `List<Integer>.remove(int)` removes by index while `remove(Object)` removes by value.

Q37. A wrapper costs an object (about 16 bytes versus 4 for an `int`), a reference in collections, an allocation on boxing outside the cache, poorer cache locality, and unboxing can throw. In hot loops, large numeric collections and counters use primitives, primitive streams (`IntStream`), arrays or specialized collections. Use wrappers where an object is required (generics, nullable columns).

Q38. `addExact`, `multiplyExact`, `toIntExact` and `absExact` throw `ArithmeticException` on overflow instead of wrapping. `floorMod` and `floorDiv` follow the sign of the divisor (useful for indexes). `Math.round` rounds half up (`-2.5` becomes `-2`) and returns a `long` for a `double`. `StrictMath` gives bit-identical results on every platform; `Math` may use faster intrinsics. `nanoTime` is monotonic, for elapsed time; `currentTimeMillis` is wall-clock time that can jump.

Q39. `Throwable` has `Error` (JVM-level problems such as `OutOfMemoryError`; not caught) and `Exception`; `Exception` splits into **checked** exceptions (`IOException`, `SQLException`; must be caught or declared) and `RuntimeException` (unchecked; programming errors such as `NullPointerException`). Wrap lower-level exceptions with the original as the **cause** (`new X(msg, e)`); `try-with-resources` records an exception from `close()` as **suppressed** on the primary one.

Q40. `Iterable` lets a type be used in a for-each loop: the compiler calls `iterator()` and loops with `hasNext()` and `next()`. `AutoCloseable` is the target of try-with-resources, so `close()` runs even on exceptions. `CharSequence` is a common read-only view of text, implemented by `String`, `StringBuilder` and others, so APIs can accept any of them.

Q41. S: one reason to change per class (small classes, composition). **O**: extend by adding code, not editing it (interfaces and polymorphism; sealed types for closed sets). **L**: subtypes must honor the parent's contract (careful overriding, `final`, contract tests). **I**: small client-specific interfaces (several narrow interfaces combined with `extends`). **D**: depend on abstractions, not concretions (interfaces and constructor injection).

Q42. Cohesion is how closely a class's members belong together; **coupling** is how much it depends on others; aim for high cohesion and low coupling. The **Law of Demeter** says to call only your own members, parameters, created objects and direct fields, so avoid `a.getB().getC().doIt()`. **Tell, don't ask** means giving an object a command (`order.cancel()`) instead of reading its state and deciding for it.

Part C: Coding and Prediction (model solutions)

C1. Reverse words

```
static String reverseWords(String s) {
    String[] parts = s.trim().split("\\s+");          // "\\s+" collapses
runs of whitespace
    StringBuilder sb = new StringBuilder(s.length());
    for (int i = parts.length - 1; i >= 0; i--) {
        sb.append(parts[i]);
        if (i > 0) sb.append(' ');
    }
    return sb.toString();
}
// reverseWords("  the   quick brown ") -> "brown quick the"
```

C2. Compress

```
static String compress(String s) {
    if (s.isEmpty()) return s;
    StringBuilder sb = new StringBuilder();
    char prev = s.charAt(0);
    int count = 1;
    for (int i = 1; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == prev) count++;
        else { sb.append(prev).append(count); prev = c; count = 1; }
    }
    sb.append(prev).append(count);                    // flush the last run
    return sb.toString();
}
// compress("aaabbc") -> "a3b2c1"
```

C3. Word frequencies

```
Map<String, Integer> freq = new HashMap<>();
for (String w : text.toLowerCase(Locale.ROOT).split("\\W+")) {
    if (!w.isEmpty()) freq.merge(w, 1, Integer::sum);
}
freq.entrySet().stream()
    .sorted(Map.Entry.<String,
Integer>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey(
)))
    .forEach(e -> System.out.println(e.getKey() + " " + e.getValue()));
```

C4. Safe parse

```
static OptionalInt parseInt(String s) {
    if (s == null || s.isBlank()) return OptionalInt.empty();
    try {
        return OptionalInt.of(Integer.parseInt(s.strip()));
    } catch (NumberFormatException e) {               // non-numeric AND out
of range both land here
        return OptionalInt.empty();
    }
}
```

C5. Refactored `ReportService`

```
record Sale(String item, long amountPaise) { }

interface ReportFormatter { String format(List<Sale> sales); }
// SRP: formatting
class CsvReportFormatter implements ReportFormatter {
    public String format(List<Sale> sales) {
        StringBuilder sb = new StringBuilder();
        for (Sale s : sales)
sb.append(s.item()).append(',').append(s.amountPaise()).append('\n');
        return sb.toString();
    }
}
interface ReportStore { void save(String name, String content); }
// SRP: storage
class FileReportStore implements ReportStore {
    public void save(String name, String content) {
        try { Files.writeString(Path.of(name), content,
StandardCharsets.UTF_8); }
        catch (IOException e) { throw new UncheckedIOException("cannot save "
+ name, e); }   // no swallowed exception
    }
}
interface Mailer { void send(String to, String subject, String body); }
// DIP: abstraction for email
class SmtpMailer implements Mailer { public void send(String to, String
subject, String body) { /* SMTP */ } }

class ReportService {
// coordinates only
    private final ReportFormatter formatter;
    private final ReportStore store;
    private final Mailer mailer;
    ReportService(ReportFormatter f, ReportStore s, Mailer m) {
this.formatter = f; this.store = s; this.mailer = m; }
    void generateAndSend(List<Sale> sales, String email) {
        String csv = formatter.format(sales);
        store.save("report.csv", csv);
        mailer.send(email, "Report", csv);
    }
}
```

The empty `catch` in the original silently lost errors; the new version throws with the cause. Each class now has one reason to change, and the service can be tested with fakes.

C6. Timed decorator

```
final class TimedPaymentMethod implements PaymentMethod {
    private static final System.Logger LOG = System.getLogger("payments");
    private final PaymentMethod delegate;
    TimedPaymentMethod(PaymentMethod delegate) { this.delegate =
Objects.requireNonNull(delegate); }

    @Override public PaymentResult pay(Money amount) {
        long t0 = System.nanoTime();                                    //
monotonic clock for elapsed time
        try {
            return delegate.pay(amount);
        } catch (RuntimeException e) {
            throw new IllegalStateException("payment failed after " + ms(t0)
+ " ms", e);   // keep the cause
        } finally {
            LOG.log(System.Logger.Level.INFO, "pay took {0} ms", ms(t0));
        }
    }
    private static long ms(long t0) { return (System.nanoTime() - t0) /
1_000_000; }
}
```

C7. Peak-hour pricing

As written, `PricingStrategy.price(Vehicle, Duration)` does not receive the **entry time**, so a decorator cannot know whether the stay started at peak time. The interface was designed too narrowly; the fix is a **parameter object** that carries all the facts a pricing rule might need:

```
record ParkingStay(Vehicle vehicle, Instant entry, Instant exit) {
    Duration duration() { return Duration.between(entry, exit); }
}
interface PricingStrategy { long price(ParkingStay stay); }
// now rules can use any part of the stay

final class PeakHourPricing implements PricingStrategy {
    private final PricingStrategy base;
    private final ZoneId zone;
    PeakHourPricing(PricingStrategy base, ZoneId zone) { this.base = base;
this.zone = zone; }
    @Override public long price(ParkingStay stay) {
        int hour = stay.entry().atZone(zone).getHour();
        long price = base.price(stay);
        return (hour >= 17 && hour < 20) ? Math.multiplyExact(price, 2L) :
price;
    }
}
// ParkingLot.exit builds new ParkingStay(vehicle, entry, clock.instant())
and calls pricing.price(stay)
```

The lesson: Open/Closed works only if the abstraction exposes enough information; evolving an interface to take a parameter object is usually better than adding more arguments.

C8. Output

```
true false true true false
5.2c1ba
```

`s1` is a constant expression (pooled), `s3` is a new object, `s4` is the pooled instance; `100` is cached and `1000` is not. The builder holds `ab1c2.5` and `reverse()` turns it into `5.2c1ba`.

Part D: Senior Level and the Whole Track (model answers)

S1. *Memory:* compact strings already halve typical log text; keep fields as `substring` results (they copy, so a small field does not retain the whole line); for **repeated tokens** (level, host, status code, user agent) with a bounded vocabulary, deduplicate with G1's `-XX:+UseStringDeduplication` (only shares the byte arrays and works after objects survive some collections) or with an application map or `intern`; avoid unbounded

`intern`. *CPU:* avoid regex in the hot path (`split` with a single plain character has a fast path; otherwise precompile `Pattern`, or use `indexOf` and `substring`), avoid `String.format` and `+` in loops, reuse a `StringBuilder` with `setLength(0)`, stream lines instead of reading whole files, and consider parsing from `byte[]` or `ByteBuffer` for the very hot path. *Measure* with JFR or async-profiler allocation profiles, `jcmd <pid> GC.class_histogram`, GC logs, heap dumps in MAT (dominators, duplicate strings), and JMH microbenchmarks; choose G1 or ZGC based on pause targets, and watch the allocation rate.

S2. Immutability prevents **time-of-check/time-of-use** bugs: a path, URL, class name or key that passed validation cannot be changed by another thread before use, so it is safe to share across trust boundaries and as a hash key. Strings cannot be wiped, may be pooled, and appear in heap dumps, core dumps and logs, so secrets belong in `char[]` that you overwrite (`Arrays.fill`), though many frameworks convert them back to `String`. `intern()` on untrusted input lets an attacker grow the string table (memory pressure, collectable but costly), and predictable hash codes enable hash-flooding of `HashMap`s (mitigated by treeification, plus input limits).

S3. The compiler rewrites boxing to `Integer.valueOf(i)`, which reads `IntegerCache.cache[i + 128]` for values in the range; the cache is built when the class initializes, with its upper bound from `-XX:AutoBoxCacheMax` (lower bound fixed at -128). C2 can **eliminate boxes** (box elimination, escape analysis) when they do not escape the compiled method, but boxes stored in collections, passed to non-inlined methods or compared by identity remain. For numeric-heavy code use primitive arrays and fields, `IntStream` and friends, `long` accumulators or `LongAdder`, and specialized collections (fastutil, Eclipse Collections); avoid `Map<Integer, ...>` in hot paths. **Project Valhalla** introduces **value classes** (objects without identity) that the JVM can flatten into arrays and fields and that can make wrapper-like types cost close to primitives; check the current JEP status for what is shipped.

S4. Use **unchecked domain exceptions** for business failures and bugs, and checked exceptions only where the caller can realistically recover and must be forced to decide. **Wrap** low-level exceptions with the cause at layer boundaries (repository to `DataAccessException`, service to domain exception) so no persistence type leaks upward, and **translate** to HTTP in one place (`@ControllerAdvice`). **Log once**, at the boundary that handles the failure, with the full stack and context ids but no secrets; do not log and rethrow at every layer. Never swallow (`catch (Exception e) {}`), do not catch `Throwable`, and keep `Error`s propagating. Use try-with-resources for every `AutoCloseable`. After catching `InterruptedException`, restore the flag with

`Thread.currentThread().interrupt()` (or rethrow). Beware `finally` blocks that `return` or throw, which hide the original exception. Do not use exceptions for ordinary control flow (stack traces are costly); use `Optional` or result types. Remember Spring's rule: **a `@Transactional` method rolls back on unchecked exceptions by default, not on checked ones** unless you set `rollbackFor`. Add retries only for transient errors, with timeouts and idempotency.

S5. (1) **Class loading** (Section 1): on first use the class loader delegates upward, loads `Order`, links it (verify, prepare statics to defaults, resolve), then **initializes** it, running superclass initialization first and then static fields and blocks once under the class-initialization lock (3.1.6, 3.2.1). (2) **`new`**: the JVM allocates from the thread's TLAB, zeroes the fields, sets the header (class pointer, lock and hash bits) (3.1.4). (3) **Constructor chain**: `super()` up to `Object`, then at each level the field initializers and instance blocks, then the body; `final` fields are assigned here and `this` must not escape (3.1.5, 3.2.2). (4) The reference is stored in a local variable. (5) **Publication** to another thread needs a happens-before edge (volatile, lock, thread-safe collection, static initializer) or, for `final` fields only, the final-field freeze guarantee; otherwise another thread may see defaults. (6) **Use**: calls are dispatched through the object's class pointer and method table (3.3.5). (7) When no GC root reaches the object it is unreachable and its memory is reclaimed by the collector (young generation first); `Cleaner` actions or unclosed resources are your responsibility. (8) The class's metadata is unloaded only when its ClassLoader becomes unreachable.

S6. Output:

```
A/B.who/A.name
A.name B.name A.name
A.show(String) B.show(Object) B.show(Object)
A.show(String) B.show(Integer)
```

Line 1: `call()` is declared in `A`. The static call `id()` is bound at compile time to `A.id`; `who()` is a virtual call, so `B.who` runs; the field `name` read inside `A`'s code is `A.name`, because fields are resolved by the class where the code is written and are not polymorphic. *Line 2:* field access uses the reference type: `a.name` is `A.name`, `b.name` is `B.name`, `((A) b).name` is `A.name`. *Line 3:* the compiler resolves overloads with static type `A`, whose visible methods are `show(Object)` and `show(String)` (`show(Integer)` is invisible). `"x"` selects `show(String)` (most specific); `B` does not override it, so `A.show(String)` runs. `o` has static type `Object`, so `show(Object)` is chosen and `B` overrides it. `5` needs boxing (phase 2) and only `show(Object)` accepts an `Integer`, so

again `B.show(Object)`. *Line 4:* static type `B` sees `show(Object)`, the inherited `show(String)` and `show(Integer)`. `"x"` picks `show(String)`. For `5`, phase 1 (no boxing) finds nothing; phase 2 boxes to `Integer`, and both `show(Integer)` and `show(Object)` apply, with `show(Integer)` more specific.

S7. Problems and fixes:

Problem	Principle	Fix
`public static Map<String, Order> orders` is global, mutable, unsynchronized	Encapsulation, thread safety	Inject an `OrderRepository`; thread-safe or database-backed
One method creates, prices, pays, emails and persists	SRP	Separate pricing, payment, notification and persistence collaborators; the service only coordinates
Six loosely typed parameters; parallel `items` and `prices[]`	Primitive obsession, data clumps	`OrderLine(sku, qty, Money)`, `Customer`, `Address` records
`double` for money	Correctness	`Money` (long paise or `BigDecimal`)
Public mutable fields (`o.id = ...`)	Encapsulation	Constructor with validation; private final fields
`if (paymentType.equals(...))` chain, no `else`	OCP, type safety	`PaymentMethod` strategy or an enum or sealed type; reject unknown types
Payment outcome ignored; order saved even if declined	Correctness	`PaymentResult` sealed type; act on each case
Email sent before persistence; no transaction	Consistency	Persist first (transaction), then publish an event or use an outbox
`sendEmail` and `saveToDb` hard-wired	DIP, testability	Interfaces `Mailer`, `OrderRepository` by constructor injection
Caller's list and array stored directly	Defensive copying	`List.copyOf` in the constructor

Problem	Principle	Fix
No validation (null email, negative price, mismatched lengths)	Fail fast	`Objects.requireNonNull`, checks in constructors
Name `OrderManager`; returns a mutable order	Naming, immutability	`CheckoutService`; return an immutable view or DTO

S8. The built-in rule kinds are a **closed set with different parameters**, so model them as a **sealed interface with records** (`PercentOff(pct)`, `BulkDiscount(minQty, amount)`, `BuyXGetY(x, y)`) and evaluate them with an exhaustive pattern `switch`, so a new built-in kind forces every evaluator to be updated. The **partner rules** are an **open set** loaded at startup, so they need an extension interface (`PricingRule`, discovered by `ServiceLoader` or as Spring beans). Combine them by adding one more permitted record, `Custom(PricingRule rule)`, which **delegates** to the open strategy; the `switch` handles built-ins directly and calls the delegate for `Custom`. An enum fails because constants are singletons and cannot hold per-instance parameters. Use composition: the engine **has** a list of rules with priorities, so rules are combined without inheritance. Also validate partner rules (timeouts, exception isolation), keep rule objects immutable, and persist rules with a type tag so they can be reloaded.

S9. *Locks* (`synchronized`, `ReadWriteLock`, `StampedLock`) work but readers contend, and a reader that makes several calls can see a **mix** of old and new values unless it holds the lock for all of them. A better design for **read-mostly** data is an **immutable snapshot with a `volatile` reference**:

```
record Config(String url, int timeoutMs, List<String> hosts) {
    Config { Objects.requireNonNull(url); if (timeoutMs <= 0) throw new
IllegalArgumentException(); hosts = List.copyOf(hosts); }
}
final class ConfigHolder {
    private volatile Config current;
    ConfigHolder(Config initial) { this.current = initial; }
    Config get() { return current; }                          // one volatile
read, no lock
    void reload(Config next) { this.current = next; }         // build fully,
then swap in one write
}
```

Readers always see a **coherent, never-changing** `Config`. `AtomicReference<Config>` is needed only if several writers compute the next value from the current one (`updateAndGet`, with a CAS loop). Safety comes from **final fields** (the freeze guarantee), `List.copyOf` and a record's lack of setters (so nothing can change after construction), and from the `volatile` write that **publishes** the new object with a happens-before edge. Build the new snapshot completely before assigning it, and never mutate a published one.

S10. **`Version`:** a `record Version(int major, int minor, int patch) implements Comparable<Version>` with validation in the compact constructor; `equals` and `hashCode` come free; `compareTo` uses a `Comparator.comparingInt(...)` chain, so ordering is consistent with `equals`; store as text (`"1.2.3"`) or three integer columns if you need range queries; embeddable in JPA. **`Currency`:** `java.util.Currency` already provides immutable, canonical ISO-4217 instances (so `==` works), or a small **enum** with code and fraction digits if the set is closed and shipped with the application; persist the 3-letter code, never the ordinal. **`OrderStatus`:** an **enum** with `canMoveTo` transition rules, persisted by name or explicit code (`@Enumerated(STRING)`); if states carried data (`Shipped(trackingNo)`), switch to a **sealed interface with records**. Natural order by ordinal is not meaningful for a status, so provide an explicit `Comparator` if needed. Records use value equality, enums use identity (`==`).

S11. Plain composition root:

```
public final class App {
    public static void main(String[] args) {
        Clock clock = Clock.systemUTC();
        OrderRepository repo = new JdbcOrderRepository(dataSource());
        PaymentMethod upi = new TimedPaymentMethod(new
UpiPayment(httpClient()));
        CheckoutService checkout = new CheckoutService(new FestiveDiscount(),
repo);
        new HttpServer(checkout, upi).start();             // the only place
that knows concrete classes
    }
}
```

Wiring happens in one place and is checked by the compiler. **Spring adds** component scanning and auto-wiring, bean lifecycle callbacks, scopes (singleton, request), AOP proxies for `@Transactional` and security, externalized configuration and profiles, auto-configuration of infrastructure, and test slices. **Testing** needs no framework: `new`

`CheckoutService(new NoDiscount(), new InMemoryOrderRepository())` with a fake `PaymentMethod` that returns a programmed `PaymentResult`, and an injected `Clock` for time; contract tests can run the same cases against the fake and the real repository.

S12. "*Always program to interfaces*": right at **boundaries and variation points** (repositories, gateways, strategies); wasteful for every internal class, value type or DTO, and an interface with one implementation and no test seam is noise. "*Prefer composition over inheritance*": a good default (3.3.6), but inheritance is right for true IS-A with Liskov substitutability, skeletal implementations and framework hooks; composition costs delegation boilerplate. "*Use records for everything*": records fit data carriers and value objects; they are wrong for JPA entities, mutable or behavior-heavy classes, inheritance, and anything that must hide its representation (components are the public API); array components and shallow immutability need care. "*Use enums instead of constants*": right for a **closed set** known at compile time; wrong when values are data (added by operators without a deploy; use a table) or when each variant needs different data (use sealed types); persist by name or code. "*Make everything immutable*": the right default for values, DTOs and configuration (3.5.4); but entities, builders, large buffers and performance-critical structures are mutable by design; confine the mutation and avoid sharing it.

End of the OOP Track

You have now completed **Part 3**: foundations and object creation (3.1), static, final, abstract and nested classes (3.2), inheritance and polymorphism (3.3), interfaces (3.4), object contracts and value design (3.5), and the `java.lang` package with the design capstone (3.6). Together with Sections 1 and 2 this gives you the core of the language.

Suggested next modules: exception handling in depth, generics and type erasure, the Collections framework internals (revisiting HashMap, ConcurrentHashMap and friends), streams and functional style in depth, I/O and NIO, and then Spring and Hibernate internals. Tell me which to take next, or ask for a mixed revision quiz over Sections 1 to 3 first.